OfflinePay Skills & Code Guide

Risheesh Pandey | Java Backend Portfolio | Milestone 6

Learn the concept, locate its implementation, run a failure scenario, and explain why the code remains correct. This guide grows with the project. It covers the wallet foundation, offline reservation and signed proofs, settlement, refresh sessions, messaging, service discovery and observability. Implemented, tested and pending capabilities are distinguished.

What this release builds

Two registered users can hold virtual INR balances, transfer money while connected to the backend, retry a request safely, and inspect their journal. The foundation uses Spring Data JPA/Hibernate and grows into a financial core, notification service, API Gateway and Eureka registry. M6 adds metrics, traces, JSON logs, k6 tests and deployment examples.

Start with one invariant

A transfer must move money once, keep total money unchanged, and leave a balanced journal. A successful HTTP response is useful only when these conditions also hold.

Action	Alice	Bob	Journal effect
Demo funding INR 1,000	1,000	0	System debit + Alice credit
Transfer INR 250	750	250	Alice debit + Bob credit
Retry same transfer key	750	250	Return existing payment; no new entries

Read and run

Follow README steps 1-3. Read AuthController, AuthService, SecurityConfig, WalletController, WalletService, Money, then Flyway migrations. Use the manual Postman lab in docs/milestone-2-part-2.md and inspect database reconciliation queries.

Available/reserved balances, rotating refresh sessions and logs are implemented. Part 2 adds device keys, signed allowances and local proofs; M3 adds settlement/deadline-gated release; M4 adds Kafka/outbox, notifications and Redis protection. M6 includes measured local and hosted PostgreSQL throughput, GitHub CI and SQL reconciliation. The final hosted appendix gives current measurements and their limits; phone exchange remains future work. No bank integration or real-money settlement is present.

Implemented skills: code map

Skill / concept	Exact implementation to read
Java records, UUIDs, enums/constants, collections	AuthController DTOs; WalletService response records; PaymentKind and EntryType enums
Java BigDecimal and integer money	common/Money.toPaise and toInr
Spring Boot, Lombok and dependency injection	OfflinePayApplication; @RequiredArgsConstructor on services; Lombok generates constructor injection
REST, JSON DTOs and validation	AuthController and WalletController request records and mappings
Spring Security; stateless JWT; issuer and expiry	security/SecurityConfig; AuthService.login
BCrypt password hashing	SecurityConfig.passwordEncoder; AuthService.signup/login
Ownership authorization	WalletController.user; WalletService.payment; PaymentOperationRepository.findVisiblePayment
Spring Data JPA, Hibernate and relationships	UserEntity, WalletEntity, PaymentOperationEntity, LedgerEntryEntity; JpaRepository interfaces
ACID and rollback	@Transactional on signup, transfer and demoTopup
Row locking and lock ordering	WalletService.transfer and lockWallet
Hibernate dirty checking and guarded debit	WalletEntity.debit/credit; managed wallet updates at transaction commit
Durable idempotency and SHA-256	WalletService.findPreviousPayment/createPayment; common/Idempotency.fingerprint
Balanced journal; immutable history	LedgerService.recordTransfer; @Immutable entities; Flyway V1/V2
Schema migrations, keys, constraints, indexes	resources/db/migration/V1...sql and V2...sql
Pagination and error contracts	WalletController.ledger; common/ApiExceptionHandler
JUnit; concurrency and integration tests	MoneyTest; PaymentFoundationTest; PostgresPaymentIT
Docker, Compose, profiles and secrets	Dockerfile; compose.yaml; Setup-Local.ps1; application*.yml
Git and CI; health checks	GitHub workflow ci.yml; Spring Boot Actuator /actuator/health

The application uses Spring Data JPA and Hibernate. Lombok generates builders/getters/constructors. Tests retain JdbcTemplate for independent SQL assertions and failure injection; application services do not use it.

Authentication and authorization

Authentication answers: who is making this request?

Signup hashes the password with BCrypt and creates an empty wallet in the same transaction. Login checks the hash and issues a signed JWT containing the user UUID as its subject. The token lives for 30 minutes.

Code location: src/main/java/com/risheesh/offlinepay/auth/AuthService.java
Lines 85-92

```
85          JwtClaimsSet claims = JwtClaimsSet.builder()
86                  .issuer(issuer)
87                  .subject(issued.userId().toString())
88                  .issuedAt(now)
89                  .expiresAt(now.plus(tokenLifetime))
90                  .id(UUID.randomUUID().toString())
91                  .build();
92          String token = jwtEncoder.encode(JwtEncoderParameters.from(
```

Authorization answers: may this user see or change this resource?

The sender is taken from the verified JWT, never from a request field. A payment lookup filters by the authenticated user's wallet. An unrelated user gets 404, avoiding disclosure of another user's transaction.

Code location: src/main/java/com/risheesh/offlinepay/payment/PaymentOperationRepository.java
Lines 11-19

```
11      @Query("""
12              select p from PaymentOperationEntity p
13              left join p.senderWallet sender
14              where p.id = :paymentId
15                and (sender.id = :walletId or p.receiverWallet.id = :walletId)
16              """)
17      Optional<PaymentOperationEntity> findVisiblePayment(
18              @Param("paymentId") UUID paymentId, @Param("walletId") UUID walletId);
19  }
```

Why stateless security?

Each request carries a bearer token. The app verifies its signature, issuer and expiry without an HTTP session. The initial HS256 secret is shared only by the backend; later separate services should use an asymmetric signing key and public verification keys. JWT signatures do not encrypt token contents.

Try it

Call GET /api/v1/wallets/me without a token: expect 401. Alter a token: expect 401. Read another pair's payment as a third user: expect 404. Never paste a JWT into public documentation.

CSRF is disabled for this bearer-header API because it does not authenticate through automatically sent browser cookies. Cookie-based authentication would need a separate CSRF design. Refresh-token rotation is now implemented; login rate limiting and mobile ownership verification are later work.

Money, DTOs and validation

Represent money exactly

Floating point can introduce rounding errors. The API accepts BigDecimal INR, converts to integer paise and stores BIGINT. INR 250.25 becomes 25025 paise. Fractional paise, zero, negatives and oversized amounts are rejected.

Code location: src/main/java/com/risheesh/offlinepay/common/Money.java
Lines 9-19

```
 9     public static long toPaise(BigDecimal inr) {
10         try {
11             long paise = inr.movePointRight(2).longValueExact();
12             if (paise <= 0 || paise > 1_000_000_000L) throw new ArithmeticException();
13             return paise;
14         } catch (ArithmeticException | NullPointerException ex) {
15             throw new ApiException(HttpStatus.BAD_REQUEST, "INVALID_AMOUNT",
16                     "Amount must be positive, at most INR 10000000, with no fractional
   paise");
17         }
18     }
19     public static BigDecimal toInr(long paise) { return BigDecimal.valueOf(paise, 2); }
```

Validate at the boundary and in the domain

Controller annotations validate required fields, phone format and password size. Money conversion validates business precision inside the service too. This matters because future jobs or consumers may call the service without going through a controller.

Code location: src/main/java/com/risheesh/offlinepay/wallet/WalletController.java
Lines 16-17

```
16     public record TransferRequest(@NotNull UUID receiverWalletId, @NotNull BigDecimal
   amount) {}
17     public record TopupRequest(@NotNull BigDecimal amount) {}
```

Same value, same request meaning

100, 100.0 and 100.00 normalize to the same 10000 paise. The idempotency fingerprint hashes normalized money, not arbitrary raw JSON formatting. This allows harmless formatting changes in a retry.

Try it

Send amounts 0, -1 and 0.001: expect 400 and no balance change. Transfer 0.01: exactly one paise should move. Send 1.000: it has no fractional paise and is accepted as INR 1.

Password input is limited to printable ASCII in this release to keep the 72-character bound equal to BCrypt's 72-byte bound. Supporting Unicode passwords requires a deliberate hashing/input design.

One atomic financial transaction

ACID is a payment requirement

Atomicity means all related changes commit or none do. A transfer inserts a payment, debits the sender, credits the receiver and writes two journal entries. Spring applies @Transactional through the service proxy. SQL failures or business exceptions roll the transaction back.

Code location: src/main/java/com/risheesh/offlinepay/wallet/WalletService.java
Lines 60-67

```
60         wallets.sender().debit(paise);
61         wallets.receiver().credit(paise);
62         // Hibernate dirty checking persists both locked wallets at commit.
63         PaymentOperationEntity payment = createPayment(
64                 wallets.sender(), wallets.receiver(), PaymentKind.TRANSFER, paise, key,
   hash);
65         ledgerService.recordTransfer(payment, wallets.sender(), wallets.receiver());
66         return toResponse(payment);
67     }
```

What happens if the receiver credit fails?

A sender debit changes a managed Java entity. Hibernate flushes entity changes inside the same transaction. SQL failures or runtime business exceptions abort the transaction. The overflow test forces receiver credit to fail after sender debit; no changed balance or payment is committed.

When is SUCCESS truthful?

The service proxy commits before returning to the controller. The HTTP response reports SUCCESS only after commit. If the response is lost, the client can retry with the original key and discover the committed payment.

Why a single deployment first?

A single PostgreSQL transaction covers both wallets here. Once services own separate databases, a REST call does not create one shared ACID transaction. We will define a ledger owner and a durable command/outbox design before splitting payment and wallet writes.

Try it

Read rollsBackWhenCreditFails in PaymentFoundationTest. Verify the sender retains its original balance and no successful operation row exists for the failed request key.

Concurrency, locks and bounded work

Two requests can see the same starting balance

Alice has INR 1,000. Two simultaneous INR 700 transfers must produce one success and one insufficient-funds result. Multiple app instances cannot rely on a Java synchronized block for shared money correctness.

Code location: src/main/java/com/risheesh/offlinepay/wallet/WalletService.java
Lines 108-114

```
108     private LockedWallets lockInOrder(UUID senderId, UUID receiverId) {
109         // Both directions acquire the same first lock, avoiding A->B / B->A deadlocks.
110         boolean senderFirst = senderId.toString().compareTo(receiverId.toString()) < 0;
111         WalletEntity first = lockWallet(senderFirst ? senderId : receiverId);
112         WalletEntity second = lockWallet(senderFirst ? receiverId : senderId);
113         return senderFirst ? new LockedWallets(first, second) : new LockedWallets(second,
    first);
114     }
```

Code location: src/main/java/com/risheesh/offlinepay/wallet/WalletRepository.java
Lines 16-18

```
16     @Lock(LockModeType.PESSIMISTIC_WRITE)
17     @Query("select w from WalletEntity w where w.id = :walletId")
18     Optional<WalletEntity> findByIdForUpdate(@Param("walletId") UUID walletId);
```

Why sort both wallet IDs?

A transfers to B while B transfers to A. If each locks its own wallet first, the transactions can wait on each other. Locking both IDs in the same deterministic order removes this common deadlock cycle. Other lock types and future flows still need consistent ordering.

Database as the concurrency authority

@Lock(PESSIMISTIC_WRITE) asks Hibernate to acquire a database row lock. Locks last until transaction completion. We query only the wallet ID before locking, so the persistence context does not retain a stale balance. WalletEntity.debit checks funds after the lock is acquired.

Bounded capacity is separate from correctness

The first Hikari pool allows 10 connections and PostgreSQL lock waits are capped at 3 seconds. These are initial demo settings, not a throughput claim. Lock contention can produce 503 and the client should retry its original key.

Read preventsConcurrentOverspending and oppositeDirectionTransfersPreserveTotal. A hot wallet remains serialized even if more application instances are added.

Durable idempotency

The retry must identify the same intended action

The client generates an Idempotency-Key before sending a transfer and reuses it when the outcome is uncertain. A new deliberate payment gets a new key. Keys are scoped to the authenticated user, and request type is part of the fingerprint.

Code location: src/main/java/com/risheesh/offlinepay/wallet/WalletService.java
Lines 116-125

```
116     private Payment findPreviousPayment(UUID userId, String key, String hash) {
117         // The actor wallet is already locked; concurrent retries wait for its commit.
118         Optional<PaymentOperationEntity> previous =
    paymentRepository.findByActorIdAndIdempotencyKey(userId, key);
119         if (previous.isEmpty()) return null;
120         if (!previous.get().getRequestHash().equals(hash)) {
121             throw new ApiException(HttpStatus.CONFLICT, "IDEMPOTENCY_CONFLICT", "Key was
    used for different request data");
122         }
123         log.info("payment_replay transactionId={}", previous.get().getId());
124         return toResponse(previous.get());
125     }
```

Concurrent duplicates

The caller holds the actor wallet lock. Only one duplicate can mutate the actor's balance at a time. The first request commits its durable payment row; the waiting request sees that row and returns the original payment. The database unique constraint is a second guard.

Why PostgreSQL rather than only Redis SET NX?

A Redis TTL can expire, Redis can restart, or a worker can crash after the debit but before storing a cached response. The committed payment row and financial effects belong in the same durable transaction. Redis may later speed up lookups, but it must not replace the durable uniqueness rule.

Contract details

Same key plus normalized same request returns the original payment. Same key with different amount, recipient or kind returns 409. Failed insufficient-funds attempts do not reserve the key in this release. Keys do not expire yet.

Try scripts/Demo.ps1. Read concurrentRetriesExecuteOnce and rejectsChangedPayloadAndOperationKind. Replay timestamps are normalized to microseconds to match database precision.

Balanced journal and reversals

A journal explains where money came from and went

Each successful operation has exactly two equal opposing entries. Wallet accounts use a liability convention: CREDIT increases the wallet projection; DEBIT decreases it. The system issuance account balances virtual demo funding.

Operation	Debit account	Credit account
Demo funding INR 1,000	DEMO_ISSUANCE: 1,000	Alice wallet: 1,000
Transfer INR 250	Alice wallet: 250	Bob wallet: 250
Reserve INR 300	Alice available: 300	Alice reserved: 300

Code location: src/main/resources/db/migration/V2__protect_journal.sql
Lines 28-29

```
28 CREATE CONSTRAINT TRIGGER operation_balanced AFTER INSERT ON payment_operation
29 DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION verify_balanced_operation();
```

Enforce the invariant at commit

A deferred PostgreSQL constraint trigger checks that the operation has exactly two journal entries whose debit and credit totals both match its amount. Deferred validation lets the application insert both entries before the check runs.

Append-only history

V2 also rejects UPDATE and DELETE on payment operations and journal entries. Schema owners can bypass triggers, so a deployment beyond the local demo needs separate migration and restricted runtime database roles. These controls are not a full accounting/audit product.

Reconciliation and recovery

For each wallet, WALLET credits minus debits match available_paise; OFFLINE_RESERVED credits minus debits match offline_reserved_paise. Run the reconciliation query in README. A future reversal creates a new operation with opposing entries and a reference to the original; it never rewrites committed history.

PostgresPaymentIT checks journal mutation rejection and unbalanced operation rejection. H2 does not run these PostgreSQL-specific triggers.

Schema, SQL, pagination and configuration

Schema evolution is explicit

Flyway applies numbered SQL migrations and records their checksums. V1 creates users, wallets, operations and journal entries. V2 protects financial history. V3 adds reserved funds/allowances; V4 adds refresh sessions. Once a migration has been applied to a shared database, change the schema with a new migration rather than editing the old file.

Code location: src/main/resources/db/migration/V1__payment_foundation.sql
Lines 26-28

```
26     UNIQUE (actor_id, idempotency_key),
27     CHECK ((kind = 'TRANSFER' AND sender_wallet_id IS NOT NULL AND sender_wallet_id <>
   receiver_wallet_id)
28         OR (kind = 'DEMO_TOPUP' AND sender_wallet_id IS NULL))
```

Constraints and indexes serve different purposes

Foreign keys prevent orphan records; unique keys prevent duplicate identities/operations; CHECK constraints bound valid states. The wallet history index supports filtered chronological journal reads. Indexing improves selected queries but adds write work, so later tuning should use query plans and measurements.

Code location: src/main/java/com/risheesh/offlinepay/ledger/LedgerService.java
Lines 33-35

```
33     public List<LedgerEntryEntity> history(UUID walletId, int page, int size) {
34         return ledgerRepository.findByWalletIdOrderByCreatedAtDescIdDesc(walletId,
   PageRequest.of(page, size));
35     }
```

Repository parameters and pagination

Spring Data derives history queries from method names; PageRequest supplies page and size. @Query defines the two custom JPQL reads. Hibernate binds parameters and generates SQL. Pagination caps page size at 100. Keyset pagination is a later history optimization.

Configuration and health

application.yml defines the service, Hikari limits, JWT settings and Actuator exposure. application-demo.yml enables virtual funding. Compose injects generated secrets from .env, and Setup-Local.ps1 never commits them.

Health is exposed without database details. There is no balance cache: this milestone reads its authoritative wallet row directly.

Tests, CI and evidence

Test behavior that could lose money

Test / tool	What it proves
MoneyTest	Exact paise conversion and rejection of invalid amounts
transfersAndBalancesJournal	Both balances move and journal sums to zero
concurrentRetriesExecuteOnce	12 simultaneous retries share one committed payment
preventsConcurrentOverspending	Two INR 700 debits cannot both spend INR 1,000
oppositeDirectionTransfersPreserveTotal	Opposing concurrent transfers preserve aggregate funds
rollsBackWhenCreditFails	A failure after deduction rolls the whole transfer back
authenticatesAndRestrictsOwnership	Bearer validation and payment ownership rules
PostgresPaymentIT	Same scenarios on PostgreSQL plus append-only and balanced-entry triggers
GitHub Actions ci.yml	Java 21 mvn verify, including PostgreSQL Testcontainers

Evidence boundary

Read docs/verification.md for the exact environment, commands and observed results. A test file or CI workflow is not a passed test. H2 behavior is useful evidence but cannot substitute for the PostgreSQL run.

Why these concurrency tests are useful

A latch releases all worker requests together. Assertions verify balances, operation uniqueness and conservation, not simply HTTP 200. These tests establish specific correctness scenarios; they do not establish 500, 1,000 or 10,000-request capacity.

Run locally

With Java 21 and Maven: mvn test for H2, and mvn verify with Docker running for PostgreSQL Testcontainers. GitHub CI runs verify on push. The Docker application build skips tests; CI and explicit verification are separate checks.

Future skills: delivery map

The following topics are planned. No implementation file is claimed until we add and verify it. Each milestone will extend this PDF with exact code references and failure exercises.

Milestone	Skills to add	Verification exercise
2: offline protocol	Java cryptography; device public keys; Ed25519; server-signed allowances; nonces; canonical signed payload; reserve balances; local verification	Exchange a signed payload without backend connectivity; reject modified amount or recipient
3: reconciliation	Allowance expiry/grace; double-spend detection; store-and-forward sync; idempotent replay; conflict decisions; reversals	Replay a receipt and submit conflicting spends; only eligible funds settle once
4: events/resilience	Kafka topics, partitions and consumer groups; transactional outbox; idempotent consumers; retry/DLT; Redis; gateway rate limiter; Resilience4j circuit breaker, timeout and bulkhead	Crash after publish; redeliver events; slow dependencies; verify no duplicate business effect
5: service boundaries	Spring Cloud Gateway; JWT public-key verification; config/discovery where useful; WebClient/OpenFeign; service data ownership; saga/recovery concepts	Inject service failure while payment is pending; recover without inventing success
6: operation/scale	Micrometer; tracing/Zipkin; correlation IDs; ELK; Kubernetes deployments, probes, secrets, HPA; k6; query plans; backpressure	Ramp load, publish p95/p99/RPS and rejected requests; reconcile money after test
7: mobile	Kotlin/Android; QR; BLE/NFC; Room/SQLite; Keystore; network detection; background sync	Both clients disconnected, then reconnect and reconcile pending receipt

JPA/Hibernate and Lombok are now implemented. Optimistic locking, CompletableFuture and caching can be introduced when a concrete use case benefits from them. Current balance updates use pessimistic locking.

Step-by-step learning checklist

Your first local session

1. Open the project in IntelliJ and confirm Java 21.
2. Run Setup-Local.ps1 and docker compose up --build -d.
3. Wait for the app startup log and run Demo.ps1.
4. Inspect the four tables in DBeaver on port 5436.
5. Run both reconciliation queries.
6. Read WalletService.transfer, WalletRepository and LedgerService; trace how entities become SQL updates.

Be able to explain these without memorizing

Why is available balance stored in paise?
Why must debit, credit and journal entries commit together?
Why do opposite-direction transfers use sorted row locks?
What happens if the server commits but the client times out?
Why does the same key with a changed amount return 409?
Why is a Redis lock alone insufficient for durable retry safety?

The offline boundary we must design next

A signed allowance proves backend authorization; it does not prove the phone has not cloned or rolled back its local state. On ordinary devices, receivers hold pending receipts with settlement risk. We will detect conflicting spends during reconciliation, cap exposure, and prohibit re-spending pending receipts.

Before implementation, define device binding, signing serialization, expiry, trusted-time limits, sync grace, reserve release, key rotation/revocation and receiver acknowledgement. The actual phone-to-phone exchange must call no backend endpoint.

Performance claims come last

10,000 virtual clients is a future experiment. Concurrent users, in-flight requests and RPS are different measurements. A successful load test must report latency, achieved throughput, errors, 429/503 rejects and financial invariants. More pods do not remove serialization on a single hot wallet.

Primary references

Spring Boot: docs.spring.io/spring-boot/3.5/system-requirements.html
PostgreSQL row locks: postgresql.org/docs/17/explicit-locking.html
Spring Security JWT: docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html
Flyway: documentation.red-gate.com/fd
Testcontainers: java.testcontainers.org/modules/databases/postgres

Annotations: what each one does

Annotation	Code location	Meaning
@Entity / @Table	All four Entity classes	Maps a Java class to an existing database table
@Id / @GeneratedValue(UUID)	Entity ID fields	Marks the primary key; Hibernate generates IDs for new rows
@Column / @JoinColumn	Entity fields	Maps values and foreign keys to column names
@OneToOne / @ManyToOne	Wallet.user; Payment wallet references; Ledger.operation	Maps database relationships using Java objects
FetchType.LAZY	Entity relationships	Loads related data when needed; map DTOs within a transaction
@Enumerated(STRING)	PaymentKind / EntryType / AccountKind	Stores readable enum names; explicit VARCHAR mapping matches V1
@Immutable	PaymentOperationEntity / LedgerEntryEntity	Hibernate does not dirty-update financial history; database triggers remain authoritative
@Lock(PESSIMISTIC_WRITE)	WalletRepository.findByIdForUpdate	Acquires a database row lock before changing money
@Transactional	WalletService.transfer / AuthService.signup	Commits related writes together or rolls them back
Propagation.MANDATORY	LedgerService record methods	Requires the existing payment transaction; no separate ledger commit
@RequiredArgsConstructor	Services	Lombok creates constructor parameters for final dependencies
@Getter / @Builder / constructors	Entities	Generates accessors/builders and the constructor required by JPA

JPA is the persistence API. Hibernate is its implementation. Spring Data JPA supplies repositories. Lombok generates Java boilerplate. These are different responsibilities, even though their annotations appear together.

We use @Getter rather than @Data on entities so password hashes are not included in generated toString output and financial history does not gain unrestricted setters.

Read the simpler transfer method

Code location: src/main/java/com/risheesh/offlinepay/wallet/WalletService.java
Lines 46-71

```
46     @Transactional
47     public Payment transfer(UUID userId, UUID receiverId, BigDecimal amount, String key) {
48         Idempotency.validateKey(key);
49         long paise = Money.toPaise(amount);
50         UUID senderId = ownWalletId(userId);
51         if (senderId.equals(receiverId)) {
52             throw new ApiException(HttpStatus.BAD_REQUEST, "SELF_TRANSFER", "Choose another
    wallet");
53         }
54
55         String hash = Idempotency.fingerprint("TRANSFER|" + senderId + "|" + receiverId +
   "|" + paise);
56         LockedWallets wallets = lockInOrder(senderId, receiverId);
57         Payment previous = findPreviousPayment(userId, key, hash);
58         if (previous != null) return previous;
59
60         wallets.sender().debit(paise);
61         wallets.receiver().credit(paise);
62         // Hibernate dirty checking persists both locked wallets at commit.
63         PaymentOperationEntity payment = createPayment(
64                 wallets.sender(), wallets.receiver(), PaymentKind.TRANSFER, paise, key,
   hash);
65         ledgerService.recordTransfer(payment, wallets.sender(), wallets.receiver());
66         return toResponse(payment);
67     }
68
69     @Transactional
70     public Payment demoTopup(UUID userId, BigDecimal amount, String key) {
71         if (!demoTopupsEnabled) {
```

What happens behind these lines?

The repository loads both wallets with database row locks. debit and credit update their managed entities. paymentRepository.save persists a new operation; LedgerService.saveAll persists its opposing journal entries. Hibernate dirty checking emits the wallet UPDATE statements when the transaction flushes.

Safe transition from the earlier JDBC bundle / earlier milestone

Keep your existing .env, IntelliJ run configuration and PostgreSQL volume. Replace pom.xml, src, docs and scripts from the updated bundle, then reload Maven. Existing V1/V2 migration files and tables are unchanged. Hibernate ddl-auto=validate checks the mapping instead of recreating tables.

For the IntelliJ run, keep DB_URL pointing to localhost:5436, set DB_PASSWORD/JWT_SECRET_BASE64 from .env, enable the demo profile, and retain -Duser.timezone=Asia/Kolkata. Follow the manual Postman lab after restart.

Milestone 2: reserve offline funds

Ab same project mein available balance ka kuch amount offline bucket mein reserve karte hain. Yeh actual offline payment abhi nahi hai; signed allowance aur device verification next part hain.

Bucket	Before	After reserve INR 300
Available	INR 750	INR 450
Offline reserved	INR 0	INR 300
Total	INR 750	INR 750

Code location: src/main/java/com/risheesh/offlinepay/wallet/WalletEntity.java
Lines 59-62

```
59     public void reserveOffline(long paise) {
60         debit(paise);
61         offlineReservedPaise = Math.addExact(offlineReservedPaise, paise);
62     }
```

Why row locking still matters

Reservation aur online transfer dono available funds consume karte hain. Dono same wallet lock use karte hain. INR 1,000 mein ek INR 700 reserve aur ek INR 700 transfer simultaneously aaye toh sirf ek succeed karega.

API and idempotency

POST /api/v1/offline-allowances accepts amount + deviceId with Bearer JWT and Idempotency-Key. Same request/key returns the existing allowance. Changed device/amount returns 409. Actor key scope topups, transfers aur reserves ke beech shared hai.

GET /api/v1/wallets/me now adds offlineReservedBalance and totalBalance. Existing availableBalance field preserved hai. Pending reserved funds online transfer mein spend nahi ho sakti.

Reservation ledger and protocol boundary

Code location: src/main/java/com/risheesh/offlinepay/ledger/LedgerService.java
Lines 38-42

```
38     public void recordOfflineReservation(PaymentOperationEntity payment, WalletEntity
   wallet) {
39         ledgerRepository.saveAll(List.of(
40                 entry(payment, wallet, AccountKind.WALLET, EntryType.DEBIT),
41                 entry(payment, wallet, AccountKind.OFFLINE_RESERVED, EntryType.CREDIT)));
42     }
```

Same wallet, different logical accounts

WALLET debit available balance reduce karta hai. OFFLINE_RESERVED credit reserved balance increase karta hai. Two entries same operation aur wallet ko refer karti hain. Total journal movement zero hai; available/reserved projection separately reconcile hoti hai.

Ledger entry	Effect
WALLET / DEBIT / 30000 paise	Available INR 750 -> INR 450
OFFLINE_RESERVED / CREDIT / 30000 paise	Reserved INR 0 -> INR 300

Read the reservation entity

OfflineAllowanceEntity stores owner, wallet, operation, allocated amount, device identifier, RESERVED status and expiry. GET by ID is filtered by owner; list is paginated. New allocations require a registered device with possession proof. The certificate snapshots its public key; registration does not prove trusted hardware.

Expiry is not automatic release

Default expiry is 24 hours. Funds do not unlock automatically; the additional sync grace defaults to 48 hours. Milestone 3 stores a sync deadline (default expiry + 48h) and exposes owner-only release after new claims close. No timer automatically unlocks funds; current device revocation blocks new settlements.

Part 2 adds a server-signed certificate for new allocations and a local signed-payment client. Pre-V5 reservations remain unsigned; signatures do not make settlement final.

Refresh tokens: rotation and replay

Login returns an access JWT in JSON and a random opaque refresh token in an HttpOnly cookie. Access lifetime stays 30 minutes. Refresh session defaults to 7 days; each rotation keeps the original session expiry.

Code location: src/main/java/com/risheesh/offlinepay/auth/RefreshTokenService.java
Lines 44-65

```
44     @Transactional(noRollbackFor = RefreshTokenReplayException.class)
45     public Issued rotate(String rawToken) {
46         String hash = hash(rawToken);
47         UUID sessionId =
   tokens.findSessionIdByTokenHash(hash).orElseThrow(this::invalidToken);
48         // Entire token family ko lock karte hain, not just one old token.
49         RefreshSessionEntity session =
   sessions.findByIdForUpdate(sessionId).orElseThrow(this::invalidToken);
50         Instant now = now();
51         if (session.getRevokedAt() != null || !session.getExpiresAt().isAfter(now)) throw
   invalidToken();
52         RefreshTokenEntity current =
   tokens.findByTokenHash(hash).orElseThrow(this::invalidToken);
53         if (current.getConsumedAt() != null) {
54             session.revoke(now);
55             log.warn("refresh_replay sessionId={} action=revoke_session", sessionId);
56             throw new RefreshTokenReplayException();
57         }
58         current.consume(now);
59         Issued issued = issue(session, now);
60         log.info("refresh_rotation_prepared sessionId={}", sessionId);
61         return issued;
62     }
63
64     @Transactional
65     public void logout(String rawToken) {
```

One-time use and session-family lock

Only the refresh-token hash is stored. rotate reads session ID, locks that session, then reads fresh token state. One use consumes the old token and issues a new one. Reuse revokes the whole session so its newer descendants cannot refresh either.

Why noRollbackFor?

Reused token par HTTP 401 return hota hai, but session revocation must commit. Narrow noRollbackFor only applies to RefreshTokenReplayException. Other failures roll back. AuthService.refresh has no outer transaction that can undo this revocation.

Serialize refresh calls in the client. If a rotated response is lost, reusing the old token can revoke the session; recovery is a fresh login.

Request IDs and honest debugging logs

Code location: src/main/java/com/risheesh/offlinepay/common/PaymentAuditLogger.java
Lines 11-15

```
11      @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
12      public void committed(PaymentCommittedEvent event) {
13          log.info("operation_committed transactionId={} kind={} amountPaise={}",
14                  event.transactionId(), event.kind(), event.amountPaise());
15      }
```

Success log belongs after commit

PaymentCommittedEvent has transaction ID, kind and amount; no entity/lazy reference is needed after commit. AFTER_COMMIT prevents a rolled-back operation from being reported as committed. This log is not a durable Kafka/outbox event; database journal remains authoritative.

Find one request across log lines

RequestLoggingFilter supplies X-Request-ID and puts it in MDC. Response header + log field share the same ID. It logs method, path, status and elapsed milliseconds, then clears MDC in finally so reused threads do not inherit another request ID.

Log file and rotation

IntelliJ: logs/offlinepay.log relative to working directory. PowerShell: Get-Content .\logs\offlinepay.log -Tail 50 -Wait. Docker: docker compose logs -f app. File rotation uses 10 MB files, 7 days history and 100 MB total cap.

Keep secrets out

Our request filter logs no bodies, query strings or auth headers. Passwords and raw access/refresh tokens are excluded. Tests check correlation, commit messages, secret-free output, safe replacement of invalid request IDs and exception status/MDC cleanup.

Manual Postman flow and upgrade

Import postman/OfflinePay.postman_collection.json. GET CSRF saves a header token. Login/refresh store AT locally; RT stays in the cookie jar and rotates through Set-Cookie. The source export contains blank token values.

Action	Request / expected result
Login	POST login with CSRF -> AT JSON + RT HttpOnly cookie
Reserve INR 300	Register device first; stable key; amount + deviceId
Check balances	Available decreases; reserved increases; total unchanged
Replay reserve	Same allowance ID; no second movement
Read history	Inspect accountKind to distinguish available/reserved entries
Refresh	No Auth; CSRF header + cookie; refresh body empty
Reuse old refresh	401; session revoked; newer descendants invalid
Logout	POST /api/v1/auth/logout -> 204; future refresh denied

Existing database stays in place

Stop app, replace source/config/docs/Postman files, keep .env and .idea. Reload Maven. Generate stable server keys using OfflineClient.main before startup. V5 databases receive V6; V1-V5 files stay unchanged. Preserve existing local keys and proofs. Old unsigned reservations remain unsigned and can be released only after their stored deadline. Keep Hibernate ddl-auto=validate; never recreate the database to apply this upgrade.

Access-token revocation boundary

Logout/replay revokes refresh access. Already-issued JWTs stay valid until expiry; immediate blacklist/revocation is not implemented. Independent login sessions are unaffected by another session logout.

Remaining requirements and evidence

Requirement	Placement
Simple annotation-based Java / Hinglish guide	Every milestone; current JPA/Lombok style preserved
Logs	Implemented now: request ID, rolling file, after-commit financial logs
Access + refresh tokens	Implemented now: hashed storage, rotation, session replay revocation
Device signing and local verification	Implemented in Part 2; see notes 22-26
Sync, conflict detection, safe release	Implemented in milestone 3; notes 29-35
Kafka/outbox, Redis limiter, circuit breaker, bulkhead, timeout	Milestone 4
OpenFeign and separate service ownership	Milestone 5
Micrometer/tracing, ELK, Kubernetes and k6	Milestone 6
Android/Room/Keystore; QR then BLE/NFC	Milestone 7

Why OpenFeign later?

Abhi services same application ke Java beans hain. Feign comes when distinct service boundaries exist. Named clients, timeouts, service authentication, request-ID propagation and idempotent retry policy will be added together; no artificial HTTP self-call is introduced now.

What has actually been verified?

Read docs/verification.md for the current passed test count and environment. H2 exercises the application, concurrent reservation/retry/refresh behavior and logs. PostgreSQL Testcontainers also includes migration preservation and V2 triggers; its execution requires Docker and remains explicitly separate.

No RPS, p95/p99 or 10K capacity claim exists. New offline reservations are not proof of disconnected payment safety. Local Git history and prepared CI are retained; GitHub publication remains deferred.

Refresh transport: cookies and CSRF

RT JavaScript se readable nahi hona chahiye. AuthController returns AccessResponse only; AuthCookies writes offlinepay_rt through Set-Cookie. Refresh/logout body empty hoti hai; cookie automatically goes to /api/v1/auth. Internal TokenResult raw RT is also JsonIgnore as a defensive boundary.

Code location: src/main/java/com/risheesh/offlinepay/auth/AuthCookies.java
Lines 16-22

```
16     public void write(HttpServletResponse response, String token, long seconds) {
17         response.addHeader(HttpHeaders.SET_COOKIE, ResponseCookie.from(REFRESH, token)
18                 .httpOnly(true).secure(secure).sameSite("Strict").path(PATH)
19                 .maxAge(seconds).build().toString());
20         response.setHeader(HttpHeaders.CACHE_CONTROL, "no-store");
21     }
22     public void clear(HttpServletResponse response) { write(response, "", 0); }
```

CSRF is a separate credential check

Cookies are ambient credentials. GET /auth/csrf returns a masked token and sets an HttpOnly XSRF-TOKEN cookie. Auth POSTs send that body token in X-XSRF-TOKEN along with the cookie. Default Spring XOR handling is retained. Missing/wrong header pair returns 403 before rotation. Bearer-only wallet/device APIs do not authenticate from RT cookies.

Observe manually

Postman: GET CSRF, login, inspect cookie manager, refresh with Body none, logout. RT is not in JSON or collection variables. HttpOnly does not hide it from DevTools. Secure is default; demo profile permits localhost HTTP. SameSite=Strict, narrow path, no Domain and no-store responses are deliberate settings.

No cross-origin CORS policy is enabled. A browser frontend needs same-origin routing or an explicitly allowed credentials origin. Existing access JWTs remain valid until expiry after logout.

Device identity and possession proof

Before Part 2 deviceId was only supplied text. DeviceService now registers user + deviceId + immutable public key after verifying a signature made by the matching private key. The Bearer JWT selects the user; a signature copied from another user/device registration does not match.

Code location: src/main/java/com/risheesh/offlinepay/device/DeviceService.java
Lines 22-31

```
22     public Device register(UUID userId, String deviceId, String publicKey, String
   signature) {
23         try { Ed25519.publicKey(publicKey); }
24         catch (IllegalArgumentException ex) { throw new ApiException(BAD_REQUEST,
   "INVALID_DEVICE_KEY", "Use Base64 X.509 Ed25519 public key"); }
25         if (deviceId == null || !deviceId.matches("[A-Za-z0-9._:-]{1,80}")
26                 || !Ed25519.verify(publicKey, OfflineProtocol.registration(userId,
   deviceId, publicKey), signature))
27             throw new ApiException(BAD_REQUEST, "INVALID_DEVICE_PROOF", "Device
   registration signature is invalid");
28         WalletEntity wallet = lockWallet(userId);
29         var previous = devices.findByUserIdAndDeviceId(userId, deviceId);
30         if (previous.isPresent()) {
31             if (!previous.get().getPublicKey().equals(publicKey) ||
   previous.get().getRevokedAt() != null)
```

What is signed?

OFFLINEPAY_DEVICE_V1, user UUID, device ID and canonical Base64 public key, separated by newline. OfflineProtocol.registration defines exact bytes. The client creates the signature locally; backend never receives its private key.

Binding and revocation

Identical registration is retry-safe. An existing device ID cannot switch keys or be resurrected after revocation. Public keys are unique. Wallet row locking serializes registration, revocation and allocation for that user. Device revoke blocks new allocations/certificate retrieval; already-offline receivers cannot instantly learn revocation.

Key possession is not hardware attestation. Android Keystore and stronger device controls remain mobile work.

Ed25519: signatures, keys and trust

Concept	Meaning / code
Private key	Signs locally. OfflineClient writes PKCS#8 Base64 in ignored local directory. Never send it to Postman/backend/GitHub.
Public key	Verifies signatures. Ed25519.publicKey accepts canonical X.509 DER Base64.
Server signature	Proves allowance integrity/origin against a pinned server public key.
Device signature	Binds payment intent to the device public key in the server certificate.
Encryption	Not provided: signed proof fields are visible.

Code location: src/main/java/com/risheesh/offlinepay/protocol/Ed25519.java
Lines 29-35

```
29     public static String sign(PrivateKey key, String payload) {
30         try {
31             Signature signer = Signature.getInstance("Ed25519");
32             signer.initSign(key); signer.update(payload.getBytes(StandardCharsets.UTF_8));
33             return Base64.getEncoder().encodeToString(signer.sign());
34         } catch (GeneralSecurityException ex) { throw new IllegalStateException("Signing
   failed", ex); }
35     }
```

Stable server keys

Generate local-keys once using OfflineClient.main keygen local-keys. ServerSigningService loads external files and tests the pair at startup. Missing/mismatched keys stop startup. No random replacement on restart: old certificates depend on stable trust. Keep backups and never use test fixtures as deployment keys.

Pin trust before going offline

Save server-key response through a trusted authenticated channel. The receiver must use its trusted key file, not a key supplied inside a sender proof. Automated key rotation/distribution remains later work. Primitive reference: RFC 8032.

Signed allowance: fixed bytes

OfflineAllowanceService.reserve validates a registered active device, locks funds, creates a UUID allowance, snapshots the device public key and signs claims BEFORE persisting the complete allowance. Payment + balances + certificate + ledger remain in one transaction.

Code location: src/main/java/com/risheesh/offlinepay/protocol/OfflineProtocol.java
Lines 24-30

```
24     public static String canonical(AllowanceClaims c) {
25         return String.join("\n", c.version(), c.issuer(), c.keyId(),
   c.allowanceId().toString(),
26                 c.transactionId().toString(), c.walletId().toString(), c.deviceId(),
   c.devicePublicKey(),
27                 Long.toString(c.allocatedPaise()), c.currency(),
   Long.toString(c.issuedAtEpochMs()),
28                 Long.toString(c.expiresAtEpochMs()));
29     }
30     public static String canonical(SignedAllowance allowance, PaymentClaims p) {
```

Fields the server protects

Version, issuer/key ID, allowance/transaction/wallet IDs, device ID and public key, allocated paise, INR and issued/expiry epoch milliseconds are signed. Fixed-order newline-delimited UTF-8 avoids JSON whitespace/property-order differences. Identifiers/keys are constrained to prevent ambiguous delimiters.

Read versus reissue

GET allowance/{id}/certificate returns the persisted signature for the owner and active device, with current validity/key checks. It does not create another reservation. Same idempotency retry gives the original allowance. Changed fields break the signature.

Legacy upgrade

V5 adds nullable certificate columns for historical reservations. All three fields must be null together or complete together. Pre-V5 funds stay reserved with no certificate; no silent binding to a newly registered key and no automatic release occurs.

Local payment creation without HTTP

OfflineClient is a normal Java main, not a Spring app. Run keygen/register/pay/verify through IntelliJ program arguments. Pay and verify need only local files and JDK crypto; stop the backend to demonstrate disconnected execution.

Code location: src/main/java/com/risheesh/offlinepay/protocol/OfflineProtocol.java
Lines 30-36

```
30     public static String canonical(SignedAllowance allowance, PaymentClaims p) {
31         String certificateHash = Idempotency.fingerprint(canonical(allowance.claims()) +
   "\n" + allowance.signature());
32         return String.join("\n", p.version(), certificateHash, p.paymentId().toString(),
33                 p.allowanceId().toString(), p.receiverWalletId().toString(),
   Long.toString(p.amountPaise()),
34                 Long.toString(p.sequence()), Long.toString(p.paidAtEpochMs()));
35     }
36 }
```

What the device signs

Payment version, certificate hash, random payment UUID, allowance UUID, expected receiver wallet UUID, amount paise, increasing local sequence and payment time. The certificate hash includes server payload + signature, binding payment to the exact certificate.

Local budget and state

The client checks the pinned certificate and its own key, then uses a cross-process file lock and per-allowance spent/sequence file. A second payment cannot accidentally exceed the locally recorded allocation. State replacement is atomic. Choose a fresh proof filename for each payment.

This is a learning client, not secure hardware. A malicious owner can copy keys or roll back files. A crash after state update but before proof write can consume local demo budget; recovery protocol is future work. Do not delete state to reclaim money.

Verification is not settlement

Code location: src/main/java/com/risheesh/offlinepay/protocol/OfflineProofVerifier.java
Lines 35-48

```
35     public static void verify(PaymentProof proof, ServerKey trusted, UUID expectedReceiver,
     Instant now) {
36         verifyAllowance(proof == null ? null : proof.allowance(), trusted, now);
37         verifyForSettlement(proof, trusted, expectedReceiver, now);
38     }
39
40     /** Checks signatures and signed payment time; DB sync deadline is enforced by
   settlement service. */
41     public static void verifyForSettlement(PaymentProof proof, ServerKey trusted, UUID
   expectedReceiver, Instant now) {
42         if (proof == null || proof.payment() == null || expectedReceiver == null) throw
   invalid("Missing payment");
43         AllowanceClaims c = verifyAllowanceSignature(proof.allowance(), trusted);
44         PaymentClaims p = proof.payment();
45         if (!PAYMENT_VERSION.equals(p.version()) || p.paymentId() == null
46                 || !c.allowanceId().equals(p.allowanceId()) ||
   !expectedReceiver.equals(p.receiverWalletId())
47                 || c.walletId().equals(p.receiverWalletId()) || p.amountPaise() <= 0
48                 || p.amountPaise() > c.allocatedPaise() || p.sequence() < 1
```

Two signatures and a receiver

Verify trusted server issuer/key/version + signature; allowance amount and validity; device signature; expected receiver; payment amount, sequence and time. API verifier derives receiver wallet from JWT. Local verifier requires expected recipient as an explicit argument.

Explicit result

VERIFIED_NOT_SETTLED means integrity and acceptance checks passed. No wallet credit/debit, journal entry or consumed-payment row is created. Stateless re-verification is allowed; it is not durable replay detection. Independent receivers cannot see every conflicting offline spend.

What remains

Milestone 3 now implements sync, durable payment/sequence deduplication, competing claims, settlement deadlines, reserve consumption and safe release. The local/API verify endpoint remains non-settling; offline-sync/payments is the financial endpoint. Device clocks can lie; signed timestamps alone do not prove true creation time. Expiry acceptance and late settlement are distinct policies.

IST output and safe schema evolution

Code location: src/main/java/com/risheesh/offlinepay/common/TimeConfig.java
Lines 23-25

```
23            public void serialize(Instant value, JsonGenerator json, SerializerProvider
   provider) throws IOException {
24
   json.writeString(DateTimeFormatter.ISO_OFFSET_DATE_TIME.format(value.atZone(IST)));
25            }
```

One instant, different display

API createdAt/expiresAt/error timestamps display ISO-8601 +05:30. Logs explicitly use Asia/Kolkata. Database timestamps stay absolute instants with UTC JDBC handling. JWT epoch seconds and signed epoch milliseconds are timezone-independent and must not be shifted by 5.5 hours.

Display	Same instant
IST	2026-10-15T21:00:00+05:30
UTC	2026-10-15T15:30:00Z

V5 only for existing V4 database

V1-V4 checksums remain unchanged. V5 creates registered_device, public-key uniqueness, and complete certificate columns. Existing balances, users, IDs and reservations remain intact. Keep .env, .idea and database volume; never drop/recreate to upgrade. Keys are a new prerequisite before app startup.

Database viewers may display another session timezone. That does not change the stored instant or API serializer.

Part 2 learning and verification map

Read / run	Observe
AuthController + AuthCookies	AT-only JSON, RT cookie rotation/clear and no-store
SecurityConfig + GET auth/csrf	Missing/wrong CSRF rejects auth POST; wallet still Bearer-only
DeviceService + Ed25519	Wrong possession key rejects; device cannot silently rebind
AllowanceService + V5	Stable signed device snapshot; historical funds unchanged
OfflineClient + OfflineProofVerifier	Create/verify with backend stopped; tampering fails
TimeConfig + application.yml	IST human timestamps; canonical epoch values unchanged

Manual example

With Part 1 balance 450 available + 300 reserved, register DEVICE-RP-002, reserve INR 100: now 350 available + 400 reserved. Download its certificate and trusted public key. Locally create INR 40 proof for Bob, then verify as Bob. Balances do not change during proof creation/verification.

Evidence and limits

See docs/verification.md for executed tests and environment. Tests cover real cookie/CSRF transport, committed replay revocation, logout, device binding, signatures, changed recipient/amount/key, expiry, legacy preservation, local cumulative budget and IST output. PostgreSQL Testcontainers checks require Docker; Java 21 and container execution are reported separately from local Java 17/H2 checks.

Follow docs/milestone-2-part-2.md step by step. No script is required for the manual learning flow. RPS/latency/capacity claims remain unmeasured.

Milestone 3: from proof to money

Bob submits the original signed payment JSON using his Bearer AT. POST /api/v1/offline-sync/payments verifies it, consumes the sender offline reserve, credits Bob available balance, records a balanced journal and stores an immutable receipt. Everything commits in one transaction. V5 walkthrough remains deferred; V6 applies automatically during upgrade.

Stage	Sender available	Reserved	Bob available
Previous lab	350	400	250
Sync payment1 INR 40	350	360	290
Retry same payment1	350	360	290
Sync payment2 INR 60	350	300	350

Three different IDs

Allowance ID identifies the allocation. Signed paymentId identifies one offline payment and its retries. transactionId identifies the server financial operation created on settlement. Same committed proof always returns the original transactionId; running pay again generates a different paymentId.

Keep verification separate

OfflineClient verify and /offline-protocol/verify still return VERIFIED_NOT_SETTLED and do not credit anything. Offline sync is a separate authenticated financial action, with database state and ownership checks.

If an old proof is beyond its stored sync deadline, do not edit timestamps or signatures. Create a fresh allowance/proof after safely releasing unused old funds when permitted.

One transaction and consistent locks

Code location: src/main/java/com/risheesh/offlinepay/settlement/OfflineSettlementService.java
Lines 62-71

```
62         LockedWallets locked = lockInOrder(certificate.walletId(), receiverId);
63         OfflineAllowanceEntity allowance =
   allowances.findByIdForUpdate(certificate.allowanceId())
64                 .orElseThrow(() -> new ApiException(NOT_FOUND, "ALLOWANCE_NOT_FOUND",
   "Allowance not found"));
65         if (!AllowanceCertificates.claims(allowance,
   signer.publicKey().issuer()).equals(certificate)
66                 || !Objects.equals(allowance.getServerSignature(),
   proof.allowance().signature()))
67             throw new ApiException(CONFLICT, "CERTIFICATE_MISMATCH", "Certificate does not
   match the stored reservation");
68
69         Optional<OfflineSettlementEntity> previous =
   settlements.findById(payment.paymentId());
70         if (previous.isPresent()) {
71             if (!previous.get().getProofHash().equals(hash))
```

Wallet locks before allowance lock

Only scalar wallet IDs are read before locking. Sender and receiver wallet rows are locked by UUID string order, the same rule as online transfers. Allowance row is locked last. Release locks its owner wallet then allowance. This avoids introducing an allowance-first/wallet-first deadlock cycle.

Code location: src/main/java/com/risheesh/offlinepay/settlement/OfflineSettlementService.java
Lines 85-94

```
85         allowance.settle(payment.amountPaise());
86         locked.sender().consumeOffline(payment.amountPaise());
87         locked.receiver().credit(payment.amountPaise());
88         PaymentOperationEntity operation = operation(locked.sender(), locked.receiver(),
   PaymentKind.OFFLINE_SETTLE,
89                 payment.amountPaise(), "sys:offline-settle:" + payment.paymentId(), hash,
   now);
90         ledger.recordOfflineSettlement(operation, locked.sender(), locked.receiver());
91         OfflineSettlementEntity settlement =
   OfflineSettlementEntity.builder().paymentId(payment.paymentId())
92
   .allowance(allowance).operation(operation).receiverWallet(locked.receiver())
93                 .sequence(payment.sequence()).proofHash(hash).proofJson(encode(proof)).sett
   ledAt(now).build();
94         // Persist, not merge: a caller-supplied paymentId must INSERT or fail uniqueness,
   never overwrite.
```

Financial invariants

Allowance settled counter increases, sender reserved balance decreases, receiver available balance increases. Ledger: sender OFFLINE_RESERVED debit / receiver WALLET credit. Payment, ledger, allowance projection and receipt share @Transactional; any failure rolls all changes back. AFTER_COMMIT success logging remains unchanged.

New operation kind is OFFLINE_SETTLE. Sender available balance does not decrease again: funds were already removed from available when reserved.

Durable retry and conflicting claims

Code location: src/main/java/com/risheesh/offlinepay/settlement/OfflineSettlementService.java
Lines 69-78

```
69         Optional<OfflineSettlementEntity> previous =
   settlements.findById(payment.paymentId());
70         if (previous.isPresent()) {
71             if (!previous.get().getProofHash().equals(hash))
72                 throw new ApiException(CONFLICT, "PAYMENT_ID_CONFLICT", "Payment ID was
   used for another proof");
73             log.info("offline_settlement_replay paymentId={}", payment.paymentId());
74             // A committed retry remains readable after expiry/release/revocation. No new
   money moves.
75             return response(previous.get());
76         }
77
78         Instant now = clock.instant().truncatedTo(ChronoUnit.MILLIS);
```

Identity plus content

paymentId is a primary key. A SHA-256 hash covers canonical signed payment plus device signature; the canonical payment includes the certificate hash. Same ID/content returns stored receipt. Changed proof with reused ID conflicts. There is no separate client Idempotency-Key header for sync.

External IDs must insert, not merge

EntityManager.persist inserts a new receipt. JpaRepository.save could choose merge for a non-null caller-controlled ID; a race against another allowance must not silently attach changed state to an existing receipt. Global ID/sequence uniqueness failures roll back and the controller translates them to 409.

Two more protections

Unique allowanceId + sequence rejects independently signed competing intents for one sequence. Different sequences can arrive out of order, but total accepted amounts cannot exceed that allowance. Another allowance on the same wallet cannot subsidize it. First successful commits win budget; a later valid signature can still be rejected.

Expiry, grace and receiver risk

Moment	What is allowed?
Before allowance expiry	New offline acceptance and online settlement
Expiry <= now < syncDeadline	No new local acceptance; previously created proof may settle
now >= syncDeadline	No new settlements; owner may release unused reserve
Exact committed retry at any later time	Read original result; no money movement

Two verifier modes

verifyAllowance/verify require current validity for accepting a new offline payment. verifyForSettlement checks signatures and signed payment time without requiring now before expiry. OfflineSettlementService enforces the persisted submission deadline using server Clock after acquiring locks.

Snapshot the policy

New allocations store expiry + configured grace (default 48h). V6 backfills old allocations with expiry + 48h and changes no signed certificate fields. Future config changes do not rewrite existing deadlines. V1 certificates do not sign this policy field; receivers need the agreed server submission policy separately.

No honest-clock guarantee

Device signatures do not prove wall-clock truth. A malicious key holder can backdate a new proof within the open grace window. Current revocation rejects all new settlements, including apparently pre-revocation payments; committed retries still return history. These are explicit receiver-risk choices, not unconditional offline safety.

Release only funds nobody can claim now

Code location: src/main/java/com/risheesh/offlinepay/settlement/OfflineSettlementService.java
Lines 112-124

```
112         if (allowance.getReleaseOperation() != null) return releaseResponse(allowance);
113         Instant now = clock.instant().truncatedTo(ChronoUnit.MILLIS);
114         if (now.isBefore(allowance.getSyncDeadline()))
115             throw new ApiException(CONFLICT, "SYNC_WINDOW_OPEN", "Funds stay reserved until
     the sync deadline");
116         long unused = allowance.remainingPaise();
117         if (unused == 0) return new Release(allowanceId, null, Money.toInr(0), "INR",
    "SETTLED", null);
118         wallet.releaseOffline(unused);
119         String hash = Idempotency.fingerprint("OFFLINE_RELEASE|" + allowanceId + "|" +
    unused);
120         PaymentOperationEntity operation = operation(wallet, wallet,
    PaymentKind.OFFLINE_RELEASE, unused,
121                 "sys:offline-release:" + allowanceId, hash, now);
122         allowance.release(operation, now);
123         ledger.recordOfflineRelease(operation, wallet);
124         committed(operation);
```

Why release waits

INR 100 allowance, INR 40 settled: remaining INR 60 stays reserved through the sync window. At deadline, new claims close even if release has not run. Owner release moves only INR 60 from reserved to available. A late INR 60 proof is rejected; the already-settled INR 40 retry returns its original result.

Ledger and retry

OFFLINE_RELEASE creates same-wallet OFFLINE_RESERVED debit / WALLET credit. It preserves total balance. A stored release operation makes the endpoint retry-safe. Fully settled allocations return amount 0 with no new financial operation. No automatic scheduler is introduced.

Legacy funds

Pre-V5 unsigned reservations cannot produce payment certificates, but may release after their backfilled deadline. Signed old certificates remain compatible because immutable claims and server signatures are preserved. Keep server/device keys and local state during upgrade.

Evidence: failures must preserve money

Scenario	Assertion
12 concurrent retries	One payment/receipt/ledger pair, one receiver credit
Competing intents / receivers	Allowance budget never negative or cross-subsidized
Same ID / changed signed proof	409, no additional movement
Same sequence / different ID	409, original consumption remains
Receiver overflow	All projections, operation/receipt and ledger roll back
Grace/deadline/release	Acceptance versus settlement remain distinct; no post-deadline new claim
Concurrent release retries	One release operation and one balance movement
Global UUID collision	INSERT/unique conflict; no save/merge money leak

Independent database inspection

Compare wallet offline_reserved_paise against SUM(allocated-settled-released) for its allowances. Compare wallet available/reserved buckets against matching ledger credits minus debits. Every operation still has exactly two equal opposing entries. Accepted proof JSON is stored without AT/RT/private keys.

Exact evidence lives with the source

docs/verification.md records test counts, runtime and unexecuted PostgreSQL/Java21 checks. H2 does not prove PostgreSQL deferred triggers or migrations. PostgresMilestone3IT includes V5-to-V6 preservation and append-only receipt tests; execution requires Docker. No RPS/p95/p99 or client-capacity claim is made.

Manual next step and reading map

Follow docs/milestone-3.md. Upgrade the same project; keep .env/.idea, database volume, local-keys, local-device-rp, local-demo and all state files. Do not rerun keygen. Set app working directory to the offlinepay folder, reload Maven, restart and check health. Flyway adds V6 only to a V5 database.

First Postman experiment

Find Bob mobile from his wallet in DBeaver. GET CSRF and login as Bob, save Bob AT separately. POST complete original payment1.json to offline-sync/payments as Bob. Check SETTLED and balances, then repeat exact body and compare transactionId. Submit payment2 and inspect allowance counters. Use an existing proof only before its stored deadline.

Code	Read for
OfflineSyncController	Authenticated sync/get/release and post-rollback collision errors
OfflineSettlementService	Sorted locks, replay, deadline/device/sequence/budget, atomic financial writes
OfflineAllowanceEntity	Counters, remaining funds and RESERVED/PARTIALLY_SETTLED/SETTLED/RELEASED
WalletEntity + LedgerService	Consume/release reserved funds with paired postings
Clock + Milestone3Test	Deterministic grace/deadline/race tests
V6 / PostgresMilestone3IT	Additive schema and immutable receipts; V5 walkthrough later

Rejected claims are logged, not persisted in a durable dispute queue. Chargebacks/reversals, adjudication, automatic release jobs, key rotation and real mobile exchange remain follow-up work. Kafka/outbox, Redis rate limits and resilience are covered in milestone 4 notes below.

Milestone 4: money first, messages reliably

A settlement can commit while the notification broker is unavailable. The solution is not to debit through Kafka. PostgreSQL still owns the synchronous balance/journal transaction; the same transaction also inserts a durable outbox event.

Failure window	What protects the result?
Crash before database commit	Money, journal and outbox all roll back
Commit succeeds; Kafka down	PENDING outbox stays in PostgreSQL
Broker accepts; relay crashes before marker	Lease expires and event may publish again
Consumer commits; offset acknowledgement lost	Duplicate event sees consumed marker; no duplicate notice
Provider accepts; HTTP response lost	Stable notification Idempotency-Key; provider must honor it

At least once, with idempotent effects

Producer idempotence does not eliminate every restart/republication duplicate. Durable inbox uniqueness handles the business effect. External SMS/push delivery is only duplicate-safe if the adapter/provider supports the stable key. We do not claim distributed exactly-once delivery.

New operations only

M4 emits notifications for new topups, transfers, reservations, settlements and releases. Old history is not backfilled, and retrying an old proof does not create a new outbox row. Kafka consumers never update wallet balances.

Transactional outbox with annotations

Code location: src/main/java/com/risheesh/offlinepay/messaging/OutboxWriter.java
Lines 20-34

```
20     @EventListener
21     @Transactional(propagation = Propagation.MANDATORY)
22     public void record(PaymentCommittedEvent event) {
23         var operation = payments.findById(event.transactionId()).orElseThrow();
24         var message = new PaymentMessage(1, operation.getId(), operation.getId(),
   operation.getKind().name(),
25                 operation.getSenderWallet() == null ? null :
   operation.getSenderWallet().getUser().getId(),
26                 operation.getReceiverWallet().getUser().getId(),
   operation.getAmountPaise(), operation.getCreatedAt());
27         try {
28             String payload = json.writeValueAsString(message);
29             entityManager.persist(new OutboxEvent(operation.getId(), payload,
   Idempotency.fingerprint(payload), operation.getCreatedAt()));
30         } catch (JsonProcessingException ex) {
31             throw new IllegalStateException("Cannot record payment event", ex);
32         }
33     }
34 }
```

Why not AFTER_COMMIT?

OutboxWriter is a synchronous Spring EventListener with MANDATORY transaction propagation. It inserts while the money transaction is still open. Serialization/insert failure aborts the financial commit. The older PaymentAuditLogger is AFTER_COMMIT and remains a log-only listener; it cannot provide durable event recording.

Versioned data, no credentials

PaymentMessage V1 carries event/transaction UUID, operation kind, participating user IDs, amountPaise and occurredAt. No AT, RT, private key, phone number or raw payment proof travels in the normal event. EventId equals operationId; the unique source operation prevents duplicate outbox creation.

Relay leases and short transactions

Code location: src/main/java/com/risheesh/offlinepay/messaging/OutboxTransactions.java
Lines 20-29

```
20     public Optional<Claim> claim(UUID id) {
21         var row = outbox.lock(id).orElseThrow(); Instant now = clock.instant();
22         boolean ready = "PENDING".equals(row.getStatus()) &&
   !row.getNextAttemptAt().isAfter(now);
23         boolean abandoned = "PROCESSING".equals(row.getStatus()) &&
   !row.getLeaseUntil().isAfter(now);
24         if (!ready && !abandoned) return Optional.empty();
25         return Optional.of(new Claim(id, row.claim(now), row.getPayload()));
26     }
27     @Transactional
28     public void finish(Claim claim, String error) {
29         var row = outbox.lock(claim.id()).orElseThrow();
```

Three separate stages

Claim in a short DB transaction; publish outside the transaction; finish in another short transaction. A random lease token fences stale workers. Only the owner of the current PROCESSING lease can mark success/failure. Network waits do not hold the database row lock.

Crash recovery

A 60-second lease can be reclaimed if a worker disappears. PENDING events use exponential retry delay capped at 300 seconds; eight failed attempts park the row as FAILED for operator investigation. Broker acknowledgement is required before PUBLISHED. A published row does not prove the consumer or user has read it.

Simple polling trade-off

Each poll inspects at most 20 candidates and rechecks readiness under a row lock. Multiple instances may inspect the same candidates but only one claims each live lease. This is a correctness-oriented portfolio implementation, not a throughput benchmark or CDC deployment.

Inbox deduplication and dead letters

Code location: src/main/java/com/risheesh/offlinepay/notification/NotificationInbox.java
Lines 32-44

```
32         var source =
   outbox.lock(message.eventId()).orElseThrow(InvalidPaymentMessage::new);
33         if (!source.getPayloadHash().equals(Idempotency.fingerprint(payload))) throw new
   InvalidPaymentMessage();
34         if (entityManager.find(ConsumedPayment.class, message.eventId()) != null) return;
35         Set<UUID> recipients = new LinkedHashSet<>();
36         if (message.senderUserId() != null) recipients.add(message.senderUserId());
37         recipients.add(message.receiverUserId());
38         for (UUID userId : recipients) {
39             entityManager.persist(new NotificationEntity(message.eventId(), userId,
   message.kind(),
40                     message.amountPaise(), clock.instant(), !webhookUrl.isBlank()));
41         }
42         entityManager.persist(new ConsumedPayment(message.eventId(),
   source.getPayloadHash(), clock.instant()));
43     }
44 }
```

One database commit for the effect

The consumer verifies event key/version and exact payload hash against its authoritative outbox, then writes distinct-user notices and consumed marker atomically. Concurrent duplicate delivery serializes on the outbox row. Offset acknowledgement follows the service transaction commit. Same-wallet operations notify one user; a transfer notifies both.

Kafka failure handling

Transient processing failures get three retries after the first attempt. Invalid messages go to a DLT. DLT publication failure must not silently advance the source. A separate DLT listener archives topic/partition/offset/payload to PostgreSQL; archive failures retry without creating a DLT loop.

Operator recovery, not public replay

Inspect and fix the cause, then requeue one authoritative outbox row or one failed provider delivery using docs/milestone-4-recovery.sql. Reuse identity; never change journal history. The consumer currently shares the monolith DB; independent service ownership is milestone 5 work.

Notifications are a separate state machine

State	Meaning
IN_APP_ONLY	Notice is stored; no external provider configured
PENDING / PROCESSING	Provider delivery queued / lease held
DELIVERED	Provider returned HTTP success; not a human-read receipt
FAILED	Eight failed delivery attempts; operator recovery needed
readAt	Independent user acknowledgement, idempotent

User APIs

GET /api/v1/notifications lists only the authenticated user, page size 1..100. POST /api/v1/notifications/{id}/read locks and checks ownership. Another user receives NOTIFICATION_NOT_FOUND. No public API exposes arbitrary user IDs, provider secrets or operational replay controls.

Actual SMS/push is a provider integration

Default configuration creates a durable in-app inbox. Optional NOTIFICATION_WEBHOOK_URL and secret connect an operator-controlled adapter. The adapter receives user ID and resolves delivery destinations. We have not sent real SMS or implemented an Android app. Existing IN_APP_ONLY notices are not suddenly sent when the URL changes.

Provider resilience without touching money

Code location: src/main/java/com/risheesh/offlinepay/notification/NotificationWebhook.java
Lines 23-31

```
23     @CircuitBreaker(name = "notificationProvider")
24     @Bulkhead(name = "notificationProvider")
25     public void send(NotificationDeliveryTransactions.Delivery delivery) {
26         if (url.isBlank()) throw new IllegalStateException("Notification webhook is not
   configured");
27         var request = client.post().uri(url).header("Idempotency-Key",
   delivery.notificationId().toString());
28         if (!token.isBlank()) request.header("Authorization", "Bearer " + token);
29         request.body(delivery).retrieve().toBodilessEntity();
30     }
31 }
```

Circuit breaker and bulkhead

Resilience4j annotations decorate the provider call. Five minimum calls, ten-call sliding window, >=50% failures opens the circuit; 30-second wait, two half-open probes. The semaphore bulkhead allows four concurrent calls and no waiting queue. Failed or rejected sends stay failed for the durable retry worker; no fake-success fallback.

Timeout and retry boundaries

HTTP connect timeout is two seconds; socket read timeout three seconds. This is not a universal wall-clock deadline for DNS or every network phase. Persistent retries run outside database transactions and cap at eight failures. A notification outage never rolls back a committed settlement.

Stable delivery identity

Every attempt sends Idempotency-Key = notification UUID. A timeout may occur after provider acceptance, so the provider must deduplicate. Circuit breaker state is local to each app process; the delivery queue and retry attempts are durable in PostgreSQL.

Redis protects requests, not balances

Code location: src/main/java/com/risheesh/offlinepay/security/RedisRequestLimiter.java
Lines 16-22

```
16      private static final DefaultRedisScript<Long> SCRIPT = new DefaultRedisScript<>("""
17          local count = redis.call('INCR', KEYS[1])
18          if count == 1 then redis.call('PEXPIRE', KEYS[1], ARGV[1]) end
19          return count
20          """, Long.class);
21      public boolean allow(String key, int limit) {
22          Long count = redis.execute(SCRIPT, List.of("offlinepay:rate:" + key), "60000");
```

Atomic counter plus expiry

One Lua script increments the count and sets expiry for the first request atomically. A crash between separate INCR and EXPIRE commands cannot leave a permanent counter. Redis is shared across instances; it never replaces PostgreSQL payment idempotency or balance locks.

Limits and identity

Defaults: 30 auth requests/IP and 120 API requests/authenticated user per 60-second window from its first request. Auth/anonymous traffic uses a hash of the socket IP; forwarded headers are not trusted. NAT users share an auth budget. Fixed windows can burst at boundaries; this is not a WAF or complete brute-force defense.

Explicit outage policy

429 includes Retry-After:60. Redis errors produce 503 RATE_LIMIT_UNAVAILABLE before protected business execution. Health bypasses the filter. This fail-closed choice reduces availability while Redis is down; clients must retain the same financial retry identity. Redis counter resets do not affect money correctness.

M4 verification and next learning steps

Evidence	Where to inspect
Money + outbox rollback and retry identity	Milestone4Test
Duplicate concurrency and stale leases	Milestone4Test / OutboxTransactions
Actual embedded Kafka + DLT delivery	KafkaNotificationTest
Provider timeout / circuit / bulkhead	NotificationWebhookTest
429 / 503 / authenticated limit keys	RequestRateLimitFilterTest
PostgreSQL V7 and real Redis Lua	PostgresMilestone4IT / RedisRateLimitIT; require Docker

Exact executed counts/runtime and unexecuted checks are recorded in docs/verification.md. Embedded Kafka is a real test broker, not proof of Docker Compose startup. No RPS/p95/p99, 10K concurrency or production availability claim is made.

Preserve the working project

Read docs/milestone-4.md. Back up your comments and merge only the upgrade ZIP if desired. V7 is additive; preserve keys, proofs, local state, .env, .idea and DB volume. Start PostgreSQL + separate Kafka9094 + Redis6381. Old receipts do not generate notifications; trigger a new demo operation.

Agreed follow-ups remain explicit

Longer grace (duration not chosen), signed syncDeadline in a compatible certificate version, sender OR receiver submission, mobile durable proofs and automatic sync/retry, 12-hour local reminder and device status updates. No INR100 fine; no early release merely because no payment reached the server. Entity/repository and V5/V6 detailed walkthrough is deferred. OpenFeign follows real service boundaries; observability/load/Kubernetes and Android follow later.

M5: one project, four applications

Application	Ownership	Local port
Financial core	Identity, wallets, ledger, outbox	9031
Notifications	Own database, inbox, provider queue, DLT	9032
Gateway	Public routing and header boundary	9030
Eureka registry	Service registration and discovery	8762

Why keep money together?

Alice sends INR40 to Bob. Sender debit, receiver credit, journal and outbox still commit in one database transaction. Splitting these writes over HTTP would require a new consistency protocol. M5 extracts notifications, whose delayed delivery can be retried independently after money commits.

Same source paths, separate runtime

The original root Maven build remains usable. services/pom.xml builds four executable JARs after the Eureka add-on. Shared notification entity/repository/domain classes are compiled from the original files through explicit include rules, preserving your learning comments. The new inbox implementation never queries core tables. Old controller, consumer and provider worker are disabled only in core split mode.

Keep the scope precise

This is a financial core plus an extracted notification service and gateway. Identity is not yet a separate microservice. One local PostgreSQL server hosts two databases and roles. Eureka registers runtime instances on8762. Gateway uses lb:// service IDs, and Feign resolves offlinepay-core through the registry. No Config Server is included.

Gateway routes and protects the boundary

Code location: services/gateway/src/main/java/com/risheesh/gateway/GatewayApplication.java
Lines 17-21

```
17         return builder.routes()
18                 .route("notifications", r -> r.path("/api/v1/notifications",
   "/api/v1/notifications/**").uri(notifications))
19                 .route("financial-core", r -> r.path("/api/v1/**").uri(core))
20                 .build();
21     }
```

More specific route first

GET /api/v1/notifications goes to9032. Transfers, login and refresh go to9031. Public Postman URL stays9030. RT/CSRF cookies and Bearer AT pass through without changing the cookie Path. Backends still enforce authentication and ownership, so bypassing the Gateway does not bypass authorization.

Example: a forged identity header

An attacker sends X-User-ID=Alice or X-Service-Token=guessed through Gateway. BoundaryFilter removes those headers. Notification ownership comes from validated Bearer authentication, not that header. Internal session and source-event endpoints have no public gateway route. Local backends bind only to127.0.0.1.

Failure is visible

Connect timeout is one second and response timeout five seconds. There is no transfer retry filter and no successful fallback body. A lost response can still follow a committed transfer; retry with the original payment idempotency key. Gateway health shows its own process health, not that every backend works.

OpenFeign: real HTTP, short interfaces

Code location: services/notifications/src/main/java/com/risheesh/notifications/CoreClient.java
Lines 8-16

```
 8 @FeignClient(name = "offlinepay-core", url = "${offlinepay.core-url:}")
 9 public interface CoreClient {
10     record EventDigest(UUID eventId, String payloadHash) {}
11     record Session(UUID userId) {}
12     @GetMapping("/internal/v1/events/{id}")
13     EventDigest event(@PathVariable("id") UUID id, @RequestHeader("X-Service-Token") String
    token);
14     @GetMapping("/api/v1/internal/session")
15     Session session(@RequestHeader("Authorization") String bearer);
16 }
```

What annotations replace

@FeignClient creates an HTTP client from the interface. @GetMapping chooses the endpoint, @PathVariable fills the UUID and @RequestHeader sends an explicit credential. Spring decodes JSON into the small record. The notification application enables this client with @EnableFeignClients.

Two credentials, two purposes

session sends the user AT to core, which validates JWT and applies Redis request protection. It returns only userId. Notifications does not receive the signing secret and cannot mint ATs. event sends a separate random service credential to the private digest endpoint; a user AT alone cannot authorize that operation.

Example with INR1

Kafka delivers event E for Alice/Bob. Notifications fetches the SHA256 digest for E payload digest from core, compares it to the received JSON, then writes notices locally. Modified amount or participant IDs change the digest and are rejected. Feign calls finish before the database transaction; a slow remote service does not hold a local notification lock.

Client connect/read timeouts are1s/2s. Feign body/header logging stays off. X-Request-ID propagates from MDC when present. No automatic financial HTTP calls or retries were added.

A local inbox transaction replaces shared locks

Code location: services/notifications/src/main/java/com/risheesh/notifications/InboxTransaction.java
Lines 25-32

```
25         int inserted = jdbc.update("INSERT INTO
   consumed_payment_event(event_id,payload_hash,consumed_at) VALUES (?,?,?) ON CONFLICT DO
   NOTHING",
26                 message.eventId(), hash, Timestamp.from(clock.instant()));
27         if (inserted == 0) return;
28         Set<UUID> recipients = new LinkedHashSet<>();
29         if (message.senderUserId() != null) recipients.add(message.senderUserId());
30         recipients.add(message.receiverUserId());
31         for (UUID user : recipients)
32             entities.persist(new NotificationEntity(message.eventId(), user,
   message.kind(), message.amountPaise(),
```

Two deliveries of the same event

Worker A inserts event E into consumed_payment_event. Worker B attempts the same unique event ID and waits for the outcome of A. If A commits, B inserts zero rows and returns. If A rolls back, B can insert and create the notices. Marker and both notices are in one local transaction, so no partial success marker can suppress missing notices.

What remains eventually consistent

The transfer can succeed while notification service is stopped. Core outbox and Kafka retain work; restarting the consumer catches up. A dependency failure retries the Kafka record without acknowledging it and may block that partition. Invalid events go to DLT; provider retry attempts remain bounded at eight and require operator action after parking.

No cross-database foreign keys

Notification event/user UUIDs are integration references. The service role owns only its own database and does not join wallets or app_user. Financial journal and authoritative outbox remain in core. Internal service authentication plus digest verification replaces the M4 direct outbox lookup.

Migration, tests and CV evidence

Preserve the two existing test transfers

Stop all Java apps. Initialize the notification database with listeners/workers disabled. Export consumed markers, notices and DLT records from old core tables; import all three into an empty destination in one transaction. IDs, readAt and provider attempts survive. Verify row counts before starting split consumers. M4 and M5 use the same group IDs and must not consume simultaneously.

Why rollback changes after new traffic

The original tables are retained, so pre-traffic rollback is simple. After split traffic, those tables are stale while Kafka offsets advanced. Stop all writers and reconcile notification tables/markers before restoring old ownership. Never enable two active owners or reset consumer offsets as a substitute for migration.

Evidence	Test / procedure
Core private contracts and split ownership	Milestone5CoreTest
Concurrent dedup, rollback, owner checks	NotificationBoundaryTest
Actual Feign request/response headers	FeignContractTest
Actual HTTP gateway routing and cookies	GatewayRoutingTest
PostgreSQL cutover + full Windows stack	docs/verification.md; user manual M5 acceptance recorded

See docs/verification.md for exact executed counts and limitations. CV can describe implemented architecture after acceptance; no measured RPS/p95/p99 is available. GitHub publishing remains pending. M4-M6 walkthrough is deferred until after M6 implementation/testing. Accepted mobile automatic-sync/reminder and signed-deadline improvements remain backlog, not completed features.

Eureka: service names become addresses

Code location: services/discovery/src/main/java/com/risheesh/discovery/DiscoveryApplication.java
Lines 7-11

```
 7 @SpringBootApplication
 8 @EnableEurekaServer
 9 public class DiscoveryApplication {
10     public static void main(String[] args) {
   SpringApplication.run(DiscoveryApplication.class, args); }
11 }
```

Registration, heartbeat, discovery

DiscoveryApplication starts the standalone Eureka registry on localhost8762. Core, notification and gateway clients register their application name, address and port. They renew leases with heartbeats. Gateway and notifications fetch the registry to discover downstream instances; the core only registers because it does not call another service.

Caller	Logical destination	Resolution
Gateway	lb://offlinepay-core	Eureka + Spring Cloud LoadBalancer
Gateway	lb://offlinepay-notifications	Eureka + Spring Cloud LoadBalancer
CoreClient Feign	offlinepay-core	Eureka + Spring Cloud LoadBalancer

Example: core moves to another port

Start a core instance on9035 with the same application name. It registers the actual port, so Gateway and Feign can discover it without editing a hardcoded downstream address. Different server ports produce different instance IDs. Eureka chooses no payment outcome and stores no money; the PostgreSQL transaction/idempotency rules still decide correctness.

Only intended routes are public

Automatic discovery routes are not enabled. Discovering the core does not expose its internal APIs through Gateway. Core still validates AT or service credentials, and notification ownership checks are unchanged. The registry is unauthenticated but bound to loopback for this local demo; production requires secured/private deployment.

Discovery failures and local startup

Eureka is not a synchronous health oracle

Registry, server response cache and client fetch intervals create a delay. New instances may take time to appear. Cached instances can keep working during a registry outage; a cold client may have no instances and return503. A crashed process can remain advertised temporarily. Self-preservation stays enabled and may retain expired leases during suspected network failure; do not promise immediate failover.

Configured address overrides are explicit

Default Gateway URLs are lb:// service names; default Feign URL is blank, so its client name is resolved by discovery. CORE_URL or NOTIFICATIONS_URL overrides deliberately bypass discovery for diagnostics and tests. Clear those overrides for Eureka acceptance. Auth, JWT cookies, database migrations and ledger transactions are not changed by this add-on.

Start in the right order

Stop the three running JARs before merging the Eureka upgrade; keep PostgreSQL/Kafka/Redis data. Rebuild all modules, then start Eureka8762, core9031, notifications9032, Gateway9030. Use a new PowerShell window for each process and import private environment only where needed. Eureka can start at64MB initial/256MB maximum heap; this is not total RSS. Monitor actual Windows memory usage on the8GB laptop.

Verification beyond a green dashboard

The supplementary verify_eureka_routing.py check starts actual Eureka, Gateway and notification/Feign JARs on random loopback ports. A small fake core API registers at another random port. It checks routing, real Feign identity lookup, invalid Bearer rejection, unavailable-instance503 and deregistration/cache refresh. It uses isolated H2, not the user database, and does not replace Windows financial acceptance. Exact results are in docs/verification.md.

Read docs/milestone-5-eureka.md for the targeted upgrade and commands. Existing notification data was already migrated; do not repeat that import. The M5 user acceptance is recorded; M4-M6 walkthrough follows M6 implementation/testing.

M6: metrics explain behavior, not money

Code location: src/main/java/com/risheesh/offlinepay/messaging/PaymentMetrics.java
Lines 12-16

```
12     @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
13     public void committed(PaymentCommittedEvent event) {
14         // Kind is a bounded enum-like value. IDs, users and amounts are never metric
   labels.
15         meters.counter("offlinepay.payment.committed", "kind", event.kind()).increment();
16     }
```

After commit ka matlab

Alice INR1 bhejti hai. Database transaction commit hone ke baad payment counter1 badhta hai. Same idempotency key retry se original payment milti hai, naya event nahi: counter dobara nahi badhta. Insufficient funds par transaction fail hoti hai: counter0 badhta hai. Functional smoke ne ye teeno cases verify kiye.

Counter is not a balance

A counter records how many committed operations this process observed. It is not the amount transferred and not a ledger total. JVM restart par reset hota hai. A crash between DB commit and telemetry recording can miss a sample. Prometheus handles counter resets for rate calculations, but accounting must still come from database/journal reconciliation.

Question	Use
Kitni transfers per second?	Rate of committed counter / HTTP endpoint count
Alice ke kitne paise hain?	Authenticated wallet API and financial DB
Same payment twice execute hui?	Idempotency operation and balanced journal checks
How many unique customers?	Not inferred from request/VU counts

Files: all four POMs add Prometheus, Brave and Zipkin dependencies. The observe profile enables a separate local management listener; normal public API port remains9030.

Queue gauges and stale monitoring

Code location: src/main/java/com/risheesh/offlinepay/messaging/OutboxMetrics.java
Lines 25-34

```
25     @Scheduled(initialDelay = 15000, fixedDelay = 15000)
26     public void refresh() {
27         try {
28             Map<String, Long> snapshot = new HashMap<>();
29             jdbc.query("SELECT status, COUNT(*) AS total FROM outbox_event GROUP BY
   status",
30                     row -> { snapshot.put(row.getString("status"), row.getLong("total"));
   });
31             counts.forEach((status, count) -> count.set(snapshot.getOrDefault(status,
   0L)));
32         } catch (org.springframework.dao.DataAccessException ex) {
33             meters.counter("offlinepay.monitoring.refresh.failures", "source",
   "outbox").increment();
34         }
```

Gauge is a current snapshot

Example:7 outbox rows PENDING,1 PROCESSING,2 FAILED. The gauges show those counts from the most recent15-second refresh. PUBLISHED means broker publish finished; it does not prove a user read a notice. Notification gauges use only the notification database for failed delivery and open DLT counts.

Why not query on every scrape?

A scrape reads cached AtomicLong values. DB queries run on a fixed schedule, so repeated metric reads do not multiply SQL queries. SQL failure leaves the last values intact and increments offlinepay.monitoring.refresh.failures. A zero queue with an increasing refresh-failure counter is not evidence the queue is healthy.

Labels must stay small

Status and payment kind have a small set of values. User ID, wallet ID, payment UUID or arbitrary requestId would create a new metric series for every value; they belong in controlled logs/traces instead. Full historical counts can become expensive on a large table: inspect PostgreSQL plans and observation overhead before production sizing.

OutboxMetricsTest verifies populated/zero counts and stale-value/failure-counter behavior after a SQL failure. Monitoring failures must not decide a payment outcome.

RPS, p95, p99 and error rate

Term	Meaning
Achieved RPS	Requests actually completed per second under the stated workload
p95 latency	About95% of observed samples finish at or below this percentile
p99 latency	Tail latency around the99th percentile; needs enough samples
Error / rejection rate	Failed or intentionally rejected requests divided by attempts
Maximum capacity	Not established by one short run or a configured VU count

Average can hide a slow minority

Hypothetical example: most requests finish quickly, but a small group waits on a hot wallet lock. Average can still look good while p99 grows. Prometheus approximates percentiles from histogram buckets; k6 computes percentiles from the request samples it observes. Do not compare numbers without matching endpoint, duration, units and workload.

Two common mistakes

Prometheus HTTP histogram values are seconds; the k6 wallet_read_ms/transfer_ms values are milliseconds. Also, one client request crosses Gateway and Core: adding both HTTP counts double-counts that work. Filter by one service and one endpoint when reporting its achieved rate.

An acceptance budget is not a result

The supplied test chooses wallet p95 below1s, transfer p95 below2s and less than1% failures. These thresholds are criteria to check, not measured project achievements. A failed threshold must remain visible in the report.

Trace ID, span ID and the outbox boundary

HTTP segment	Tracing role
Gateway receives inbox request	Server span in a new or valid incoming trace
Gateway calls Notifications	Client/server child spans share traceId
Notifications calls Core using Feign	Client/server spans share the same traceId
Core validates identity	Core log carries traceId and propagated requestId

Request ID and trace ID have different jobs

X-Request-ID is a readable correlation label accepted only in the allowed8-64 character format or generated as a UUID. It can be supplied twice by a client, so it is not a payment identity. Brave creates/manages trace and span IDs. W3C traceparent carries trace context across HTTP; a trace contains several timed spans.

Actual verification

The M6 smoke starts all four JARs with Eureka and checks one shared traceId across Gateway, Notifications and the actual Core. A local Zipkin-format collector receives the exported spans. It tests transport/propagation, not the real Zipkin search UI.

Durable work starts later

The financial transaction writes an outbox row. The original HTTP trace context is not stored in that row. A scheduled relay later starts another trace; Kafka observations can propagate that later context to its consumer. Use transaction/event UUIDs to correlate these two stages. Do not claim the complete delayed outbox flow is one uninterrupted original HTTP trace.

The instrumented RestClient.Builder preserves HTTP observation support for optional webhook calls. Feign body/header logging remains off.

Readable console, searchable JSON logs

Code location: observability/shared-resources/logback-spring.xml
Lines 13-18

```
13   <encoder class="net.logstash.logback.encoder.LogstashEncoder">
14    <timeZone>Asia/Kolkata</timeZone><includeContext>false</includeContext>
15    <includeMdcKeyName>requestId</includeMdcKeyName><includeMdcKeyName>traceId</includeMdcKe
   yName><includeMdcKeyName>spanId</includeMdcKeyName>
16    <customFields>{"service":"${APP_NAME}"}</customFields>
17   </encoder>
18  </appender>
```

One event, structured fields

The file encoder writes service, level, timestamp, message and allowed MDC keys requestId/traceId/spanId as JSON fields. This lets Kibana search one request without guessing how to split a text line. Console output remains readable while you learn. The shared resource is packaged into each JAR.

IST is presentation, not a changed instant

A log timestamp such as2026-10-22T01:20:00+05:30 and the corresponding UTC instant describe the same moment. API/log display uses Asia/Kolkata. PostgreSQL/client sessions can display UTC; changing the display timezone does not move a payment in time.

Keep sensitive values out

No request body, password, AT, RT or cookie is intentionally logged. MDC uses an allowlist. The controlled smoke checks its known AT/password/JWT secret are absent in captured JSON files. That check does not guarantee arbitrary future code cannot leak data: keep new log statements deliberate.

Retention scope

Each application rotates10MB files with7-day/100MB history cap. The active file adds some extra space. Elasticsearch indexes have their own disk lifecycle; this sample does not automatically delete old search indexes. Logstash reads active JSON .log files, not gzip archives, and may ingest duplicates after rotation/replay. Logs never authorize a second payment.

Observe in stages on an 8 GB laptop

Stage	Run / inspect
First	Postgres + Kafka + Redis, four capped JARs, direct management endpoints
Then	Native host Prometheus; optional short Zipkin trace session
Separate exercise	Stop apps, inspect stored JSON using optional ELK
Later on suitable machine	Kubernetes cluster and actual HPA/load acceptance

Heap cap is not total memory

-Xmx256m bounds Java heap, not metaspace, thread stacks, direct buffers or all RSS. Docker/WSL, browser, IDE and OS also need RAM. The earlier hang happened under severe memory pressure; do not infer that a green container health check means there is room for another JVM. Stop normally if memory approaches sustained90%.

Loopback management

The observe profile listens on9131(core),9132(notifications),9130(gateway),9762(discovery). Health, info and Prometheus are the exposed diagnostics. env is not exposed. Native Prometheus on the host can scrape127.0.0.1 without broadening the listeners. Optional Zipkin UI uses9412; ELK uses9201/5602 to avoid common conflicts.

Alerts are visible, delivery is separate

Five Prometheus rules cover target-down, failed outbox, open DLT, gauge-refresh failure and HTTP5xx ratio. Prometheus can evaluate them; external delivery needs Alertmanager/contact configuration. No SMS/push integration was added by this milestone. ELK is a local loopback-only demo, not an internet service.

k6 creates the traffic automatically

Code location: benchmarks/mixed.js
Lines 44-51

```
44 export default function (data) {
45   // Exactly 7 wallet reads / 3 fresh transfers per 10 iterations; one hot sender.
46   const transfer = __ITER % 10 >= 7;
47   const headers = { Authorization: 'Bearer ' + data.alice.token, 'Content-Type':
   'application/json',
48     'X-Request-ID': 'load-' + run + '-' + __VU + '-' + __ITER };
49   let response;
50   if (transfer) {
51     headers['Idempotency-Key'] = 'load-' + run + '-' + __VU + '-' + __ITER;
```

You do not click Postman hundreds of times

k6 runs the request loop. setup creates fresh demo users, performs cookie/CSRF login and adds virtual money in isolated databases. Business calls use ATs. Each fresh transfer gets a unique idempotency key; the replay test sends the same key concurrently. The corrected exporter allowlists report fields to omit setup_data and tokens. The original exporter accidentally included test ATs; page62 explains the fix.

Know the workload

The mixed loop targets7 wallet reads and3 fresh INR1 transfers per10 iterations, sharing one sender/receiver. That is deliberately a hot-wallet contention test. A VU is a concurrent test worker, not a unique customer. A closed model sends the next request after the previous one completes; it does not promise a fixed arrival rate.

Start small and preserve the normal demo

Follow benchmarks/README.md: new core/notification databases, dedicated Kafka topics, RedisDB1 and test-only raised but enabled limits. Run1VU/30s, then5 and10 only if resources permit. Normal120/min protection would reject this workload intentionally. Record trace sampling, JSON logging, JVM caps and same-host k6 contention.

scripts/read_load_summary.py prints endpoint p95/p99, accepted writes, errors and threshold outcomes. It does not infer maximum capacity. Restore normal startup in fresh shells after testing.

Reconcile money after traffic stops

Code location: benchmarks/reconcile-core.sql
Lines 3-10

```
 3 SELECT COUNT(*) AS negative_wallets FROM wallet WHERE available_paise < 0 OR
   offline_reserved_paise < 0;
 4 SELECT COUNT(*) AS unbalanced_operations FROM (
 5   SELECT operation_id FROM ledger_entry GROUP BY operation_id
 6   HAVING COUNT(*) <> 2 OR SUM(CASE WHEN entry_type='CREDIT' THEN amount_paise ELSE
   -amount_paise END) <> 0
 7 ) problems;
 8 SELECT COUNT(*) AS duplicate_keys FROM (
 9   SELECT actor_id,idempotency_key FROM payment_operation GROUP BY actor_id,idempotency_key
   HAVING COUNT(*) > 1
10 ) problems;
```

Example: one transfer, two requests

A fresh Alice has INR100 and Bob0. Two concurrent identical INR1 payment requests must produce Alice99, Bob1, one operation and two balancing ledger entries. Matching HTTP transaction IDs are useful, but verify the database too. Reusing that key for INR2 must reject the changed payload.

Whole-run invariants

Require zero negative balances, zero unbalanced operations, zero duplicate actor/key groups and zero duplicate event/user notices. The total available+reserved wallet money equals virtual demo issuance. After the outbox and Kafka consumer drain, consumed-event count equals committed operation count, and notices equal distinct participants per operation. A topup has one participant; a transfer normally has two.

Fast API, growing queue is not sustained delivery capacity

The default relay polls at most20 rows with a2s delay between batches; the hosted experiment selects100ms explicitly. A transfer burst can commit faster than notices are produced. Record queue growth and drain time separately from API p95. FAILED rows/DLT need review; do not silently exclude them to make the report look successful. Both core and notification SQL files are read-only checks on isolated load databases.

PostgreSQL EXPLAIN ANALYZE/BUFFERS queries are supplied for wallet/inbox lookups. Plans and tuning remain pending on realistic PostgreSQL data; no unsupported index-performance claim is made.

Kubernetes and CPU autoscaling prerequisites

Resource	Purpose
Deployment	Replicas, resources, non-root container, startup/liveness/readiness
Service	Private stable entry point to pods
Secret references	External DB/JWT/service credentials and stable signing-key mount
HPA autoscaling/v2	CPU target60%, min1/max3; Metrics Server prerequisite

60 percent of what?

CPU request is200m per pod. A60% utilization target corresponds to about120m average CPU against that request, not60% of the laptop CPU. HPA needs Metrics Server data and available cluster capacity. A maximum of3 replicas is a configured bound, not evidence that scaling occurred or that throughput tripled.

Replica identity

Every pod receives SERVICE_IP from status.podIP. Eureka instance IDs include application name, IP and port, so replicas on the same port do not overwrite one another. Core rows/idempotency keys still serialize conflicting money writes; more replicas cannot remove the intentional lock on one hot wallet.

Application-only example

deploy/kubernetes/apps.yml contains13 schema-validated resources. It requires reachable PostgreSQL/Kafka/Redis/Zipkin, real Secrets, images, TLS ingress for secure browser cookies and a configured cluster. It does not provision those systems. Core keys stay external/stable. EmptyDir logs are ephemeral; add a cluster shipper for persistence.

No Kubernetes/HPA runtime acceptance or production-availability claim has been made. Read deploy/kubernetes/README.md before applying; do not start a cluster alongside the current8GB stack.

Verified evidence and what remains

Executed	Result
Earlier authoring unit run	100 tests; zero failures/errors/skips
Actual four-JAR metrics/tracing smoke	PASS; H2 + real Redis, Eureka, HTTP/Feign
k6 functional smoke and replay	59 business requests; no business errors; concurrent retry identity verified
Management behavior	Private metrics; Redis failure readiness503/liveness200
Prometheus/Kubernetes static checks	5 valid rules;13 valid resources

User PostgreSQL checks now pass

On the user Windows/Java 21 laptop, 1/5/10-VU runs of 30 active seconds passed their thresholds and both databases reconciled after drain. At 10 VUs: 163.10 mixed requests/s, 333.32 ms transfer p95, zero request failures. Pages 61-62 give the actual scope. This is achieved local throughput, not maximum capacity.

Runtime checks still separate

The earlier authoring-host 100-test run used a Java 17 override without Docker integration. Hosted Java 21 CI has since passed all 184 unit/integration cases, including PostgreSQL and Redis suites, plus discovery and observability smoke checks. See docs/ci-validation.md and the hosted appendix. Kubernetes/HPA runtime scaling remains unmeasured; deployment manifests are not evidence of actual scaling.

M7 and accepted backlog remain visible

Android Kotlin/Room/Keystore, QR exchange then BLE/NFC, durable proof queue and automatic network sync remain ahead. Longer sync window duration is undecided; signed syncDeadline needs a compatible protocol update. Sender OR receiver submission,12-hour local reminder and status updates remain accepted follow-ups. No INR100 fine and no early release just because the server has not seen a proof. The source and measured hosted evidence are now published in github.com/risheeshpandey/Offlinepay.

Measured PostgreSQL load results

Source: user-provided k6 v1.3.0 JSON and SQL outputs, 22 October 2026 IST. Windows, Java 21.0.11, PostgreSQL 17.6, Kafka 3.9.1, Redis 7.4.11. All services and k6 share the 8 GB laptop. These runs were executed by the user with guided commands.

30 active seconds	1 VU	5 VUs	10 VUs
Business requests	1,448	3,399	5,131
Accepted transfers	433	1,015	1,533
Mixed requests/s	46.10	107.00	163.10
Transfers/s	13.79	31.95	48.73
Read p95 / p99, ms	26.19 / 44.97	30.71 / 69.35	19.91 / 34.42
Transfer p95 / p99, ms	71.59 / 102.18	193.66 / 457.66	333.32 / 481.82
HTTP/business failures	0% / 0%	0% / 0%	0% / 0%

Read the number with its workload

About 70% wallet reads and 30% fresh INR 1 transfers share one sender/receiver. Thus 163 mixed requests/s is not 163 payment transfers/s; the observed transfer rate was 48.73/s. k6 rates include setup time in the denominator, about 31-32s total. Each run adds 9 setup HTTP calls. VUs are test workers, not distinct customers.

Controlled settings, remaining uncertainty

Redis limits stayed enabled at test-only 10,000 auth/min and 100,000 API/min. JSON logging stayed enabled; trace sampling was 0. Core heap cap 384 MiB, other services 256 MiB each. No peak CPU/RAM, soak, saturation or multi-instance/HPA test was captured. Better read latency at 10 VUs does not prove optimization; short sequential runs can differ in warm-up/cache/resource conditions.

Before 10 VUs: about 1.14 GiB physical memory free. Docker snapshot: Redis 9.715 MiB, Kafka 731.1 MiB, PostgreSQL 116.2 MiB. These are pre-run snapshots, not peak readings. docs/m6-load-results.md/json preserves measurements without credentials.

Reconciliation, queue drain and CV evidence

Final cumulative invariant	Observed result
Demo topups / transfers	4 / 2,982
Wallet money = issued money	40,000,000 paise = INR 400,000
Published / consumed events	2,986 / 2,986
Expected / actual notices	5,968 / 5,968
Negative wallets / unbalanced operations	0 / 0
Duplicate keys / notices / open DLT	0 / 0 / 0

Why 5,968 notifications?

Each of 4 topups has 1 recipient; each of 2,982 transfers has 2 participants. Therefore 4 + (2 x 2,982) = 5,968 notices. Replay checks and earlier runs are included. The standalone replay test also confirmed two concurrent identical requests created just one INR 1 transfer; changed payload with the same key was rejected.

The API finished before background work

The first core query showed 368 pending and 1 processing event. A later check showed all 2,986 published and consumed with all 5,968 notices. The backlog drained, but elapsed drain time was not timestamped. No sustained notification delivery rate can be derived from API RPS. Financial success and eventual delivery completeness are separate checks.

Export only the evidence you need

Code location: benchmarks/mixed.js
Lines 68-74

```
68 export function handleSummary(data) {
69   // k6 includes setup_data (with access tokens) here. Export only report fields.
70   const report = {
71     root_group: data.root_group,
72     state: data.state,
73     metrics: data.metrics,
74     offlinepayMetadata: { runId: run, vus: options.vus, duration: options.duration,
```

The first exporter serialized the entire k6 object, including setup_data with ATs. The corrected exporter includes only checks, state, metrics and generated metadata. Three real-k6 regression checks verify omitted secret fields, preserved results and unmodified input. Never copy the original tokens into evidence or Git.

A defensible CV measurement

Load-tested a Spring Boot/PostgreSQL wallet backend at 10 virtual users: about 163 requests/sec on a 70% read / 30% transfer workload, 333 ms transfer p95 and zero request failures in a 30-second local run. This does not establish maximum capacity or production scale.

Hosted testing: skills and code map

The GitHub workflow starts a disposable Linux stack, measures real requests through Gateway, and checks the resulting PostgreSQL records. It does not connect to the Windows laptop or its normal demo database. The source, workflow and retained reports are versioned together.

Skill	Implementation and purpose
GitHub Actions / Java CI	.github/workflows/ci.yml: 184 Java unit/integration cases plus Eureka/Feign and metrics/tracing smoke checks.
Disposable infrastructure	compose.capacity.yaml and CapacityRun.start: unique Compose project, fresh databases, generated secrets and ephemeral loopback ports.
Automated traffic	benchmarks/mixed.js: 7 reads / 3 fresh transfers per ten iterations, one sender; ci-mixed.js records the business interval.
Pass/fail policy	scripts/run_capacity_ci.py api_result: require native k6 success, full duration and the original numeric SLOs.
Financial checks	benchmarks/capacity-core.sql: balanced operations, conserved issuance, per-wallet journal agreement and unique keys.
Eventual delivery checks	capacity-notifications.sql and wait_delivery: every event consumed, expected notices present, no duplicates or open DLT.
Regression and privacy tests	test_capacity_ci.py and ci-summary-test.js: failed/short runs cannot pass; setup_data and access tokens are excluded.

A concrete example

An INR 1 transfer needs one committed operation, a debit and credit, one outbox event, and two participant notifications. A retry with the same key must return that existing operation. Fast HTTP responses alone cannot prove these records agree.

Thresholds remain HTTP/business failures below 1%, wallet-read p95 below 1000ms and transfer p95 below 2000ms. Eventual delivery and SQL reconciliation must also pass. A failed search stage stays visible and no higher stage is started.

Confirmed hosted measurement

Report status: CONFIRMED_OBSERVATION. Measurements below are actual fixed-VU confirmations, not configured targets. API traffic, four JVMs, PostgreSQL, Kafka, Redis and k6 share one runner.

Measurement	Best confirmed throughput	Highest confirmed VUs
Virtual users	300	300
Business seconds	180.51	180.51
Mixed requests/s	754.29	754.29
Completed requests	136,156	136,156
Accepted transfers	40,536	40,536
Read p95, ms	659.60	659.60
Transfer p95, ms	744.73	744.73
Transfer p99, ms	953.05	953.05
Post-traffic drain, s	320.25	320.25
HTTP / business failures	0.0000% / 0.0000%	0.0000% / 0.0000%
Accepted transfers/s	224.56	224.56

Both columns refer to the same confirmation run. Do not add their request, transfer or notification counts together.

Deployment: 4 assigned CPUs; 15,989MiB RAM; Java 21; HTTP over loopback through Gateway. Core/notification pools 10/3; outbox batch 20 and poll 100ms. Test-only Redis budgets stay enabled. About 70% reads / 30% INR 1 online transfers share one sender wallet.

Tested boundary: first failing search level 500 VUs; read p95 1,101.25ms and transfer p95 1,423.04ms. The failure remains in the report. Untested intermediate VU counts are not an exact maximum.

What this proves

The reported confirmation passed its API rules and financial/eventual-delivery checks. Notification queues can grow during traffic and clear afterward; drain time is not part of API latency. This does not prove continuous notification throughput at the API request rate, phone-to-phone delivery, multiple backend replicas, or a universal maximum.

Evidence: https://github.com/risheeshpandey/Offlinepay/actions/runs/39769439816
Retained report: benchmarks/evidence/extended-39769439816/report.json

Find the outbox bottleneck in code

Code location: src/main/java/com/risheesh/offlinepay/messaging/OutboxTransactions.java
Lines 17-18

```
17     public List<UUID> ready() { return outbox.ready(clock.instant(), PageRequest.of(0,
   20)); }
18
```

Code location: src/main/java/com/risheesh/offlinepay/messaging/OutboxRelay.java
Lines 17-30

```
17     @Scheduled(fixedDelayString = "${offlinepay.messaging.poll-ms:2000}")
18     public void poll() {
19         if (!workersEnabled) return;
20         for (var id : transactions.ready()) {
21             var claim = transactions.claim(id);
22             if (claim.isEmpty()) continue;
23             String error = null;
24             try { publisher.publish(claim.get()); }
25             catch (Exception ex) {
26                 error = ex.getClass().getSimpleName();
27                 log.warn("outbox_publish_failed eventId={} type={}", id, error);
28                 if (ex instanceof InterruptedException) Thread.currentThread().interrupt();
29             }
30             transactions.finish(claim.get(), error);
```

Batch size and polling delay are separate controls

ready() selects at most 20 rows. poll() claims, publishes and finishes them one at a time. With the default 2000ms delay after each batch, even instant work cannot exceed 10 events/s for one worker. Actual database and Kafka work lowers that rate further. Faster payment commits therefore create a growing queue.

Measured diagnosis, recorded change

The first hosted run passed API/financial checks but failed notification drain. A separate deployment selected --offlinepay.messaging.poll-ms=100 while keeping batch 20, the financial code, error/latency rules and 600s drain budget. In the initial tuned run, its 200-VU/180s confirmation passed and the queue cleared after 285.12s. The latest higher-range measurements are on page 64. Different runner CPU models and warmup conditions mean this is not a precise causal API-speedup comparison.

Highest observed is not an unconditional maximum

The runner stops after a failed search stage and selects throughput and concurrency candidates separately. Each selected VU count needs a longer passing confirmation. A prior backlog must clear before confirmation. Read docs/capacity-results.md for the tested boundary, every failure and the actual hardware; VUs, mixed RPS and payment transfers/s answer different questions.